Proyecto · Retrieval & Memory Lab

IA Generativa Avanzada y Sistemas Multi-Agente · UTDT

Grupos de hasta **4**. Enunciado publicado el **lunes 2026-10-05**, entrega el **domingo 2026-10-25**.

---

0 · En una pantalla

Qué vas a construir. Un asistente de RAG **con memoria** sobre un corpus en español que elegís y limpiás vos. Corre en una GPU T4 gratuita de Colab, solo con modelos abiertos (`qwen3:8b` servido por Ollama). La pregunta del proyecto: **cómo repartir una ventana de contexto finita entre lo que dice el corpus y lo que se dijo en la conversación.**

Qué entregás. Un zip en el campus con notebooks (con sus salidas), un informe, un registro de decisiones y los artefactos que nos permiten recalcular tus tablas. Detalle en §5.

Las etapas.

Etapa	Qué hacés	Carácter	Necesita
0 · El corpus como decisión	Elegís un corpus y lo auditás con números	a tu criterio (§2)	--
1 · De la fuente a los documentos	Convertís la fuente en documentos limpios, con ids estables	**obligatoria**	el corpus elegido
2 · La vara de medir	Escribís y congelás el conjunto de evaluación	**obligatoria**	los ids de sección de la Etapa 1
3 · Recuperación sobre tu corpus	Construís el índice y medís qué piezas se ganan su costo	a tu criterio	Etapas 1 y 2
4 · Respuestas fundamentadas	Armás prompts dentro de un presupuesto de tokens verificado y puntuás las respuestas	a tu criterio	*cualquier* configuración de la Etapa 3
5 · La conversación	Medís con qué buscar y qué historial conservar en una ventana compartida	a tu criterio: **el corazón del lab**	el ensamblador de la Etapa 4 y los guiones de la Etapa 2
6 · Memoria entre sesiones	Construís una memoria que sobrevive a la sesión y la comparás con reproducir el historial	a tu criterio	el *loop* de la Etapa 5 (no sus experimentos) y los escenarios de la Etapa 2
7 · Una segunda lectura	Un juez LLM calibrado y/o tu loop reimplementado en un	opcional, recomendada	cualquier conjunto de respuestas guardadas (7a);

Etapa	Qué hacés	Carácter	Necesita
	framework		el loop de las Etapas 5-6 (7b)

Lo que no se negocia.

- Las Etapas 1 y 2 son obligatorias, y el conjunto de evaluación se **congela antes de que exista ningún resultado**.
- Solo modelos abiertos, sin APIs pagas, y un `num_ctx` explícito en cada llamada al LLM (§6).
- Un registro de decisiones desde el primer día (§2.4).
- Si un modelo redactó ítems de evaluación, guiones o sondas, decís cuáles y cómo los verificó una persona (§2.3).

La ruta que recomendamos. Recortá desde el final, nunca desde el medio. Y desconfiá del recorte que parece más seguro: una grilla completa de la Etapa 3 y nada después. **Tres filas bien acotadas en la Etapa 3 y una Etapa 5 real hacen un proyecto mejor** (§2.2).

Cómo se evalúa. Seis criterios (§5.2): priorización, diseño experimental, comprensión conceptual, lectura de resultados, contribución original, reproducibilidad. Cero puntos por tipear código: se evalúa qué elegiste hacer, por qué, y qué leíste en tus resultados.

Cómo leer este documento.

- **§1** es la idea, en una página. La **Puesta en marcha** que sigue te deja con el código y el entorno listos. **§2** te pide un presupuesto y un plan *antes* de escribir código: leelo antes de arrancar.
- **§3** tiene las etapas, con una estructura común: *Para qué · Entra / sale · Pasos · Trampas · Preguntas para el informe*.
- **§4-§7**: artefactos, entregables y rúbrica, logística, y todos los números concretos (tamaños, presupuestos, cantidades).
- El **Apéndice A** define cada término técnico (*brazo*, *gold*, *oráculo*, *contención*...). El **Apéndice B** reúne los papers y los números de referencia que vas a citar.

---

1 · La idea

Este trabajo toma el pipeline de recuperación que construimos para RAG en el curso, lo saca del corpus para el que se construyó y lo apunta a un corpus que elegís y limpiás **vos**. Después le pide que sostenga una conversación -- dentro de una sesión y entre sesiones -- en una T4 gratuita.

La tesis. El modelo no recuerda nada entre llamadas: en cada turno solo puede usar lo que tu sistema puso en su ventana de contexto. Por eso la recuperación y la memoria son **la misma decisión** -- qué entra en una ventana finita -- tomada frente a dos fuentes:

- **un corpus** que escribió otra persona, y que podés limpiar antes de empezar;
- **una conversación** que tu sistema va escribiendo sobre la marcha, y que tenés que limpiar mientras sucede, con el mismo modelo que está hablando.

El pipeline de RAG del curso corría sobre un corpus que ya había sido limpiado para él. Acá ese trabajo es tuyo, y es donde están la mayoría de las sorpresas.

La línea de base honesta: "si entra, pegalo". Un recuperador tiene que ganarle a pegar los documentos enteros en el prompt; una memoria tiene que ganarle a reproducir el historial completo. Buena parte de este lab consiste en averiguar cuándo lo logran.

El recorrido, con cada paso medido contra "pegalo y listo":

1. un corpus que limpiaste (Etapas 0-1);
2. una vara de medir que congelaste (Etapa 2);
3. recuperación medida con esa vara (Etapa 3);
4. respuestas que podés verificar (Etapa 4);
5. una ventana compartida por pasajes y conversación (Etapa 5);
6. memoria que sobrevive a la sesión (Etapa 6).

Las Etapas 3-4 reusan lo que construimos para RAG. Las Etapas 5-6 no suponen nada previo: los fragmentos de código de este enunciado y las fuentes del Apéndice B alcanzan para arrancar.

Puesta en marcha

Hacela antes de leer §2: te da el código, el entorno y la estructura de archivos sobre los que se apoya todo lo demás. El README tiene el detalle paso a paso; acá va lo que tenés que saber para no perder tiempo.

1 · Conseguí el código. El material vive en https://github.com/solidgoldmagickarp/projecto_rag.

```
git clone https://github.com/solidgoldmagickarp/projecto_rag.git
cd projecto_rag
```

Para trabajar en grupo, hacé un fork o creá un repositorio nuevo con una copia de este, y que todos clonen *ese*. El fork y el token de Colab para un repositorio privado están en la sección "Trabajar desde el fork de tu grupo" del README. Corré todo desde la raíz del repo.

2 · Elegí dónde corrés.

- **Colab (T4)** es el entorno de destino y el que usamos para evaluar. Abrí el notebook, elegí la T4 *antes* de correr nada, y corré la primera celda: clona el repo, instala los requirements y, con `USE_DRIVE = True`, guarda `artifacts/` en tu Drive. El orden exacto está en el README.
- **Local, en CPU**, para las corridas `SMOKE_TEST` (§7.4). Necesitás Python 3.10 o más nuevo (Colab usa 3.12 o más):

  ```
  python -m venv .venv
  .venv\Scripts\Activate.ps1          # Windows (PowerShell); en Linux/macOS:
  source .venv/bin/activate
  pip install -r requirements.txt
  python verify_setup.py
  ```

  `verify_setup.py` chequea Python, paquetes, acceso a HuggingFace y el servidor de Ollama. Para las etapas que llaman al LLM, instalá Ollama (https://ollama.com/download) y bajá un modelo chico

(`ollama pull qwen3:1.7b`); `python verify_setup.py --model qwen3:1.7b` lo prueba de punta a punta. Detrás de un proxy corporativo o del campus, el README explica cómo arreglar los errores de certificados.

3 · Corré `00_corpus_audit` primero. Es la Etapa 0 resuelta sobre python-docs-es (`00_corpus_audit.py`, con su notebook `00_corpus_audit.ipynb`). En Colab, abrí el notebook con el botón del README; en local, `python 00_corpus_audit.py`. Sirve de modelo para auditar tu corpus y además es la prueba de que tu entorno funciona.

4 · Cómo se organiza tu código.

- Cada etapa es un `.py` con marcadores `# %%`, y el notebook se *genera* a partir de él: editás el `.py` y corrés `python tools/py_to_notebook.py`. Los notebooks que subís al campus (§5.1) son los que corriste, con sus salidas.
- Para que `tools/py_to_notebook.py` y `tools/run_stages.py` los encuentren, nombrá los archivos de tus etapas `01_documents.py`, `02_eval_set.py`, `03_retrieval.py`, `04_grounded_answers.py`, `05_conversation.py`, `06_memory.py` y `07_second_read.py`. También podés trabajar directamente en notebooks: cualquiera de las dos está bien, pero decí cuál en tu informe.
- Antes de commitear, corré `python tools/check_repo.py`: rechaza notebooks con salidas, así que en git van el `.py` y el notebook limpio, y el notebook *con* salidas va solo en el zip.

5 · Dependencias propias. Lo que tu grupo necesite además de `requirements.txt` va en un archivo **`requirements-group.txt`** en la raíz del repo. La primera celda de cada notebook lo instala en Colab si existe; en local, `pip install -r requirements-group.txt`. Copiá ahí el bloque de tu corpus de `requirements-corpora.txt` (por ejemplo `pymupdf` para PDFs, `py7zr` para StackOverflow) y, para la Etapa 7b, las líneas que uses de `requirements-extras.txt`. Nada de eso se instala solo.

---

2 · El alcance lo definís vos

Acá hay más trabajo del que cualquier grupo va a terminar bien. Esa es la idea: **cómo construir este proyecto es decisión tuya**, y esa decisión también se evalúa -- qué llevaste hasta el final, qué recortaste o hiciste al mínimo, cómo gastaste tu tiempo y tu cómputo.

- Un proyecto que hace **menos, pero hasta el final, con una razón explícita para el recorte**, saca más que uno que hace **todo a medias**.
- Un proyecto con menos etapas pero con un punto de vista genuino -- que conecta lo que construiste con otra cosa que sabés, o que cuestiona un resultado en vez de reportarlo -- puede sacar más que una implementación impecable y completa sin nada de sus autores adentro.

2.1 · Primero, estimá tu presupuesto

Antes de escribir código, calculá cuánto tiempo tiene realmente tu grupo y qué te compra una sesión gratuita de Colab. Este lab tiene costos que un pipeline de RAG de un solo turno no tiene:

- **Qué entra en la T4 a la vez.** El generador, el embedder y el reranker comparten 15 GB (§7.2 tiene la cuenta).
- **Cuántas pasadas de embeddings vas a pagar.** Una por cada variante de chunking o de prefijo que pruebes.

- **Cuántas llamadas al LLM cuesta cada experimento.** Es una multiplicación: ítems × turnos × llamadas por turno (respuesta, reescritura, actualización del resumen, escritura en memoria) × brazos. Hacela antes de correr: una grilla que parece razonable puede ser diez horas de generación.
- **Cuánto lleva escribir el conjunto de evaluación.** Más de lo que pensás, y no se puede saltear.

2.2 · Después, escribí un plan

Decidí qué etapas vas a llevar hasta el final, cuáles al mínimo y cuáles vas a saltear. Ese plan es la primera entrada de tu registro de decisiones (§2.4), y al final lo vas a comparar con lo que realmente pasó.

Prerrequisitos duros. La columna *Necesita* de la tabla de §0 te dice cuánto te cuesta cada recorte. Las Etapas 1 y 2 alimentan todo. **Recortá desde el final, no desde el medio.**

La ruta que recomendamos. Recortar las Etapas 5-7 es una elección legítima. Pero el recorte que más discutiríamos es el que parece más seguro: una grilla completa de la Etapa 3 y nada después. La Etapa 3 repite un protocolo que ya corrimos en el curso; la Etapa 5 es donde vive la pregunta de este proyecto. **Tres filas bien acotadas en la Etapa 3 y una Etapa 5 real hacen un proyecto mejor.** Si parás en la Etapa 4, defendelo en el registro de decisiones.

Repartir el trabajo. Cada etapa consume la salida de las anteriores: acuerden los nombres de archivo (§4) antes de que alguien empiece. Una vez congelado el conjunto de evaluación (Etapa 2), cuatro líneas de trabajo pueden correr en paralelo, cada una contra un *stub* (una versión provisoria y simple) de la anterior:

- la grilla de la Etapa 3;
- el ensamblador de la Etapa 4, sobre un recuperador denso simple;
- el loop de la Etapa 5, sobre un primer borrador del ensamblador;
- el almacén de la Etapa 6, sobre un primer borrador del loop.

Enchufar después el mejor recuperador cuesta volver a correr una vez: presupuestalo. Y separá roles: quien construya la Etapa 6 no debería ser el único autor de sus sondas (Etapa 2).

2.3 · Sobre el uso de IA

Usala: para escribir código, para depurar, para discutir. Pero fijate lo que le hace a este trabajo. Si un modelo puede producir la implementación en una tarde, la implementación ya no es lo que distingue tu trabajo. Lo que queda es lo que no puede hacer por vos: decidir qué valía la pena construir, juzgar si un resultado es creíble, notar lo que nadie te pidió que buscaras, y sostener una opinión que puedas defender. Esa parte escribila vos; es la parte que leemos.

- En tu informe, contá brevemente cómo la usaste.
- **Regla específica de este lab:** si un modelo redactó alguno de tus ítems de evaluación, guiones de conversación o sondas de memoria, decí cuáles y cómo los verificó una persona. Un conjunto de evaluación escrito por un modelo mide si tu sistema está de acuerdo con ese modelo.

2.4 · El registro de decisiones (entregable)

Junto con tus notebooks y tu informe, llevá un registro continuo en `decision_log.md`:

- el presupuesto que estimaste al principio y tu plan;
- qué probaste;

- qué recortaste y por qué (tiempo, VRAM, llamadas, beneficio conceptual).

Terminar en un lugar distinto de tu plan está bien y es esperable: decilo, y decí qué te hizo cambiar de opinión. **Cualquier edición del conjunto de evaluación congelado es una entrada del registro.**

2.5 · El análisis extra es bienvenido

Si aparece algo más interesante que las etapas -- una comparación que no sugerimos, un modo de falla que merece un párrafo, un vínculo con otro paper o con otra materia --, seguilo y contanos.

Un punto de partida posible: ¿tu ventana es tan grande como dice?

1. Tomá 10-20 de tus preguntas.
2. Rellená el contexto con pasajes irrelevantes de tu propio corpus hasta unos 2K, 4K, 8K y 16K tokens. Usá un único `num_ctx` para los cuatro, lo bastante grande para el más largo más su reserva de salida (por ejemplo, 20.480), y defendelo como pide §7.2.
3. Poné el pasaje con la respuesta al principio, en el medio y al final.
4. Graficá con qué frecuencia sobrevive la respuesta, y compará tu curva con los números de RULER del reporte de Qwen3 y con *Lost in the Middle* (Apéndice B.2).

¿Cuál es la ventana más grande en la que realmente confiarías, y cuánto te habría costado usar más?

---

3 · Las etapas

Cada etapa sigue la misma estructura (algunas no necesitan todas las partes):

- **Para qué**: el problema que resuelve la etapa.
- **Entra / sale**: qué artefactos lee, cuáles escribe, y qué va al informe.
- **Pasos sugeridos**: una guía, no una receta. Lo que hagas distinto, justificalo en el registro de decisiones.
- **Trampas**: errores que *no tiran ninguna excepción*. El código corre y la tabla sale mal. Leelas antes de correr.
- **Preguntas para el informe.**

Los fragmentos de código son **ilustrativos**: usan nombres que vienen de tu propio código (`body`, `url`, `docs`, `ART`, `messages`, ...) y no se ejecutan tal cual. Adaptalos.

Etapa 0 · El corpus como decisión

Para qué. El pipeline de RAG del curso corría sobre un corpus elegido para que los problemas de recuperación aparecieran sin ningún trabajo sobre los datos: documentos largos, estructura explícita, identificadores oficiales, referencias literales dentro del texto. El tuyo no lo eligió nadie. Y el techo del corpus es el techo del pipeline: ninguna técnica posterior recupera lo que los documentos no contienen.

Entra / sale. Entra: el menú de corpora.md (o un corpus tuyo) y la auditoría resuelta `00_corpus_audit` (cómo correrla: Puesta en marcha, punto 3). Sale: la tabla de auditoría de tu corpus y un borrador de `corpus_card.md`.

Las ocho preguntas de la auditoría. Son las mismas que responde `00_corpus_audit`, y las mismas que corpora.md pide para traer un corpus propio:

1. ¿Está la respuesta ahí (y en qué idioma)?
2. ¿El texto está limpio? ¿Qué dejaría pasar un loader ingenuo sin ningún error?
3. ¿Tiene estructura por la que se pueda cortar?
4. ¿Cada documento tiene un identificador estable, que sobreviva a una nueva descarga?
5. ¿Hay metadatos (una fecha, una versión, un estado), y son correctos?
6. ¿Cambia, o tiene versiones? ¿Cómo te enterarías?
7. ¿Se puede armar un conjunto de evaluación sobre él?
8. Y una que no es una propiedad sino un hecho: **¿cuán parecidos son los documentos entre sí?** Los vecinos confundibles son lo que hace difícil la recuperación.

Las preguntas 1-7 son "las siete propiedades"; la 8 es "la similitud".

Pasos sugeridos.

1. **Elegí un corpus** de corpora.md, o traé el tuyo (argumentado frente a las mismas ocho preguntas).
2. **Auditalo** como `00_corpus_audit` audita python-docs-es, y agregá lo que esa auditoría deja afuera:
   - cantidad de documentos;
   - distribución de largos **en tokens del generador** (la auditoría resuelta solo estima el total);
   - marcadores de estructura, y qué tan regulares son;
   - qué identifica a un documento, y si ese id sobrevive a una nueva descarga;
   - cobertura de metadatos: la proporción de documentos con una fecha / versión / estado utilizable;
   - una estimación de casi-duplicados sobre una muestra (la auditoría resuelta solo cuenta el solapamiento exacto de párrafos entre versiones);
   - diez documentos crudos leídos por una persona, de principio a fin.
3. **Completá la tabla de la auditoría**: propiedad · medido · **qué etapa posterior paga por cada "no"**.
4. **Conseguí tres números antes de comprometerte:**
   - el tamaño del corpus frente a la ventana que te podés permitir;
   - un **chequeo puntual a libro cerrado**: cinco preguntas al generador, sin recuperación;
   - el tamaño de la tarea de datos sucios que corpora.md describe para tu corpus.
5. **Escribí un borrador de `corpus_card.md`**: fuente y versión fijada, licencia y atribución, regla del subconjunto, defectos conocidos, qué pone a prueba bien el corpus.

Si elegiste python-docs-es, la auditoría resuelta es **donde tu Etapa 0 empieza, no donde termina**. Mide las dos rutas de entrada a ese corpus y no elige ninguna: esa elección es tuya.

Preguntas para el informe.

- ¿Qué propiedades de las siete no cumple tu corpus, cuánto te cuesta la octava (la similitud), y en qué etapa vas a pagar por cada una?
- ¿Tu subconjunto entraría en una ventana con el `num_ctx` que te podés permitir? Y si entrara, ¿para qué construir recuperación?
- ¿Cuánto de tu corpus es plausible que el generador ya haya leído, y cómo vas a distinguir *recuperación* de *recuerdo*?
- ¿Qué hace que dos de tus documentos se puedan confundir, y querés conservar esa propiedad?

Etapa 1 · De la fuente a los documentos *(obligatoria)*

Para qué. Todas las etapas posteriores cuelgan del registro de documento: las etiquetas gold apuntan a él, los chunks se cortan siguiendo sus secciones, los filtros leen sus metadatos, las citas nombran su id. El corpus del curso traía ese registro de regalo. Acá lo construís vos, y la mayor parte del trabajo es decidir qué es un *documento*.

Entra / sale. Entra: la fuente cruda (nunca en git). Sale: `artifacts/<profile>/documents.jsonl` (el contrato `schema.Document`) con su manifiesto, y `corpus_card.md`. Va al informe: la corpus card y los números de la limpieza.

Pasos sugeridos.

1. **Obtené y fijá la fuente** -- commit, rama, versión de DOI, fecha de dump, o una lista de URLs con fecha de descarga y sha256 -- y registrala en un manifiesto.
2. **Extraé** según el formato. Para las tablas elegí una de tres: mantenerlas, linealizarlas en oraciones "encabezado: valor", o reemplazarlas por una fuente estructurada oficial. Revisá una muestra a ojo.
3. **Normalizá**: Unicode NFC, espacios en blanco, guiones de corte de línea, encabezados y pies de página repetidos, y markup. Del markup sacá la sintaxis pero conservá los nombres que lleva: `:meth:`Path.resolve`` debería quedar `Path.resolve`.
4. **Asigná ids estables.** El `doc_id` sale de la identidad propia de la fuente; el `section_id` es `doc_id` + un camino estructural. Nunca el orden de las filas, nunca un nombre de archivo de descarga.
5. **Extraé metadatos** -- fecha, versión, estado o vigencia, orador, URL -- y *validalos contra el cuerpo* (ver la trampa "metadatos que mienten"). Ojo con el estado: `DocMetadata.status` acepta solo `current`, `superseded`, `deprecated`, `draft` o `unknown`, y cualquier otro valor (un "in force" copiado de la fuente) es un error de validación. Mapeá el estado de tu fuente a uno de esos, documentá el mapeo en la corpus card, y si querés conservar el valor original guardalo en un campo extra (`DocMetadata` acepta claves adicionales, por ejemplo `source_status`).
6. **Encontrá los casi-duplicados** sobre una muestra (Jaccard de shingles o coseno de embeddings), decidí una política y registrala en los metadatos (`version`, `canonical_id`, `duplicate_of`, ...).
7. **Escribí** `documents.jsonl`, su manifiesto y `corpus_card.md`.
8. **Medí qué cambió la limpieza**: por regla, cuántos documentos tocó y cuántos caracteres eliminó; y un diff antes/después sobre diez documentos.

```
import re
import unicodedata

from schema import Document, DocMetadata, Section, documents_hash,
validate_documents
from store import build_or_load, write_jsonl

def normalize(text: str) -> str:
    # NFC importa: algunos extractores emiten acentos descompuestos ("e" +
U+0301). Para BM25 y
    # para el puntaje por coincidencia exacta, "información" pasa a ser dos
cadenas distintas --
    # ningún error, solo fallos.
    text = unicodedata.normalize("NFC", text)
    # Colapsá espacios y tabs DENTRO de cada línea, pero conservá los saltos de
línea: los
    # encabezados y marcadores por línea son lo que usás para cortar secciones. Y
```

```
normalizá ANTES
    # de calcular los offsets de las secciones (`start`/`end` apuntan al texto ya
normalizado).
    lines = [" ".join(line.split()) for line in text.splitlines()]
    return re.sub(r"\n{3,}", "\n\n", "\n".join(lines)).strip()

# Con estos dos contadores, el paso 8 ("medí qué cambió la limpieza") se reduce a
un print.
CLEANING_HITS: dict[str, int] = {}    # regla -> documentos que cambió
CLEANING_CHARS: dict[str, int] = {}   # regla -> caracteres que eliminó

def apply(rule: str, fn, text: str) -> str:
    out = fn(text)
    if out != text:
        CLEANING_HITS[rule] = CLEANING_HITS.get(rule, 0) + 1
        CLEANING_CHARS[rule] = CLEANING_CHARS.get(rule, 0) + len(text) - len(out)
    return out

doc = Document(doc_id="3.13/library/pathlib", title="pathlib",
               text=body, sections=
[Section(section_id="3.13/library/pathlib#basic-use", title="Uso básico",
                                        level=2, start=0, end=812)],
               metadata=DocMetadata(version="3.13", status="current",
source_url=url, lang="es"))

# El hash del corpus depende de los documentos, no del orden en que los
escribiste: va en
# cada manifiesto de índice, así un índice puede decir a partir de qué corpus se
construyó.
corpus_sha = documents_hash(docs)
validate_documents(ART / "documents.jsonl").show().raise_if_failed()
```

Trampa: dos documentos que son el mismo documento. Ejemplos: informes mensuales consecutivos armados con una misma plantilla; la misma página en tres versiones de la documentación; una ley y su texto actualizado posterior; una pregunta y su duplicado marcado por un moderador. Cada uno tiene su propio `doc_id`, con texto casi idéntico.

El problema: una métrica a nivel de documento cuenta como fallo el *contenido correcto* recuperado bajo el id de un hermano. Una técnica que cae en la otra versión parece peor, y un filtro de versión que resuelve el problema parece que perjudica.

Qué hacer: encontrá los casi-duplicados acá y registrá cuáles son

- **versiones**: dan respuestas distintas ("en 3.12" vs "en 3.13");
- **copias**: son intercambiables.

Esa distinción decide cómo etiquetás en la Etapa 2. Una pregunta que no fija una versión tiene un *conjunto* gold (vale cualquiera de los documentos que la responden). Una pregunta que sí la fija necesita que el filtro de versión sea parte del sistema.

Trampa: metadatos que mienten. Un campo puede estar mal con total seguridad: un "status: in force" en una norma cuyas propias notas dicen que fue derogada, una fecha que es un valor de relleno. Si alguna etapa

posterior filtra por un campo, verificá ese campo contra el cuerpo en una muestra y reportá la tasa de desacuerdo.

Preguntas para el informe.

- ¿Qué cambió la limpieza, en números, y qué regla cambió más?
- Nombrá un defecto que un loader ingenuo habría dejado pasar sin ningún error, y cómo lo encontraste.
- ¿Cómo definiste un documento y una sección, y qué se rompe si una etapa posterior no está de acuerdo?
- ¿Tus `doc_id` sobrevivirían a una descarga nueva el mes que viene?
- ¿Cómo trataste los casi-duplicados, y a qué te compromete eso en el conjunto de evaluación?

Etapa 2 · La vara de medir *(obligatoria)*

Para qué. La medición más útil del pipeline de RAG del curso no usaba ningún modelo: preguntas escritas a mano, cada una etiquetada con el documento que la responde. Acá la vara tiene que llegar más lejos: preguntas de seguimiento que solo tienen sentido en contexto, y hechos dichos en una sesión y preguntados en otra. Nadie armó eso para tu corpus en español (no encontramos ningún benchmark así; Apéndice B.5). Junto con la Etapa 1, esta etapa **alimenta todo**, y se **congela antes de ajustar cualquier decisión del pipeline contra ella.**

Entra / sale. Entra: `documents.jsonl` (para los ids gold). Sale: `eval/single_turn.jsonl`, `eval/dialogues.jsonl`, `eval/scenarios.jsonl` y `eval/FROZEN.json`, todos commiteados. Va al informe: la descripción del conjunto y su chequeo de contaminación.

Qué vas a escribir: tres conjuntos.

Conjunto	Contrato	Cantidad (§7.3)	Lo usa
Preguntas de un solo turno	`schema.EvalItem`	≥ 30: ≥ 3 fuera del corpus; ≥ 5 de identificador, si tu corpus tiene identificadores; ≥ 5 de metadatos/temporales	Etapas 3-4
Guiones de conversación	`schema.Dialogue`	8-15 guiones de 3-6 turnos	Etapa 5
Escenarios multi-sesión	`schema.Scenario`	3-6 escenarios de 2-4 sesiones, con 5-10 sondas cada uno	Etapa 6

Pasos sugeridos.

1. **Preguntas de un solo turno**: al menos 30, escritas a mano y parafraseadas, a partir de una lista de *temas* (no con el pasaje abierto; ver la primera trampa).
    - **Categorías** (el vocabulario de `schema.CATEGORIES`): `topical` (temáticas); `identifier` (nombres exactos, números, códigos); `metadata` (una versión, una fecha, un período); y `out_of_corpus` (fuera del corpus).
    - **Fuera del corpus**: al menos 3, y **adyacentes** -- un mes que no tenés, una función que no existe en tu versión --, no "¿quién ganó el Mundial de 1986?". Decidí *ahora* qué cuenta como respuesta

correcta a una de ellas (negarse, o un "eso no existe en tu versión" fundamentado), y después puntuá cada brazo con esa regla. En el schema llevan `answerable: false`, sin `gold` ni `expected_verdict`; si no hay ninguna, `freeze_eval` se niega a congelar (no habría cómo medir la abstención). Un "no" que el corpus sí respalda, con gold y veredicto, es un ítem con respuesta, en otra categoría.
   - **Gold** a nivel de **documento y sección** en todos los ítems con respuesta; como *conjunto* cuando las copias son intercambiables (Etapa 1).
   - Una `answer_key` (clave de respuesta) corta donde exista, y en cada pregunta de sí/no su `expected_verdict` (ver la trampa "una clave de respuesta no puede decir que no").
   - Las preguntas de identificador se pueden *derivar* de identificadores que aparecen en exactamente un documento: derivalas, no las escribas.

2. **Chequeá la contaminación** con el fragmento de abajo, y corré ya una recuperación **solo con BM25**. Reescribí los ítems marcados.

3. **Guiones de conversación** (`schema.Dialogue`), de 3-6 turnos. Cada turno lleva:

   - una **etiqueta de dependencia** (`depends_on`, vocabulario de `schema.DEPENDENCIES`). El primer turno es `standalone`; los de seguimiento son `coreference` (correferencia), `ellipsis` (elipsis), `topic_shift` (cambio de tema), `return` (volver a un tema anterior) o `unanswerable` (sin respuesta);
   - gold y clave de respuesta (salvo los turnos `unanswerable`, que llevan `answerable: false`, sin gold ni veredicto);
   - la **respuesta guionada del asistente** (`assistant`). En la Etapa 5 todos los brazos reciben estas respuestas como historial, en lugar de las suyas, para que vean exactamente la misma conversación;
   - una **reescritura autónoma escrita a mano** (`oracle_rewrite`): la pregunta del turno, reformulada para que se entienda sin historial. Es la fila oráculo de la Etapa 5.

4. **Escenarios multi-sesión** (`schema.Scenario`):

   - una persona *ficticia* cuyos datos importan para tu corpus: una versión que usa, una región que sigue, una fecha "a la cual" pregunta;
   - sesiones con **timestamps simulados**;
   - sondas: preguntas hechas en una sesión posterior, cada una etiquetada con la habilidad que evalúa (`schema.ABILITIES`: `extraction`, `multi_session`, `temporal`, `knowledge_update`, `abstention`, `preference`) y con las sesiones que contienen la evidencia (`evidence_sessions`);
   - **requisito:** cada escenario tiene al menos un hecho que **cambia** y uno que **nunca se dijo**.

5. **Separá roles**: quien construya la Etapa 6 no debería ser el único autor de sus sondas.

6. **Separá dev y test.** Cada fila es `"test"` por defecto. Marcá un puñado de ítems por categoría con `split: "dev"`: eso es lo que ajustás y mirás mientras construís. Reportá sobre `test`.

7. **Congelá** contra el corpus completo:

```
schema.freeze_eval("eval", environment.artifacts_dir(False) /
"documents.jsonl")
```

Eso escribe `eval/FROZEN.json`. **Commitealo antes de que exista ningún resultado.** Si lo congelás en Colab, queda en un disco que se borra al desconectarse: descargalo y commitealo desde tu máquina.

A partir de ahí, `schema.load_eval("eval", documents_path=...)` rechaza un conjunto editado, o un corpus que cambió por debajo. Si después editás el conjunto, la edición es una entrada del registro de decisiones, y cada fila puntuada antes de ella se vuelve a correr o se descarta. (Los ítems smoke viven aparte, en `eval/smoke/`, y se cargan con `require_frozen=False`; ver §7.4.)

```
import re
import Stemmer   # PyStemmer

stem = Stemmer.Stemmer("spanish").stemWord
STOP = set("de la que el en y los se del las un por con no una su para es al lo
como más".split())

def content_words(text: str) -> list[str]:
    # La MISMA normalización que usa tu canal léxico: un chequeo que tokeniza
distinto que
    # BM25 mide un solapamiento distinto del que infla a BM25.
    return [stem(w) for w in re.findall(r"\w+", text.lower()) if w not in STOP]

def contamination(question: str, gold_text: str, n: int = 5) -> dict:
    q, g = content_words(question), content_words(gold_text)
    overlap = sum(w in set(g) for w in q) / max(len(q), 1)
    grams = lambda ws, k: {tuple(ws[i:i + k]) for i in range(len(ws) - k + 1)}
    shared = max((k for k in range(len(q), 0, -1) if grams(q, k) & grams(g, k)),
default=0)
    return {"overlap": round(overlap, 2), "longest_shared_ngram": shared, "flag":
shared >= n}
```

Trampa: las preguntas escritas leyendo el pasaje miden copia. Si escribís una pregunta con el pasaje gold abierto, importás sus palabras raras y sus frases exactas. BM25 las encuentra, la recuperación densa no necesita generalizar, y "el híbrido le gana al denso" o "el reranker ayuda" pasan a ser artefactos de *cómo se escribieron las preguntas*. No hay ningún error en ningún lado; la tabla simplemente mide otra cosa.

Qué hacer: escribí a partir de temas, medí el solapamiento y reescribí lo marcado. Las preguntas deliberadamente literales van en su propia categoría (`identifier`), donde la coincidencia literal es justamente lo que se mide.

Trampa: un modelo escribiendo tu vara de medir. Si un modelo redacta preguntas a partir de un pasaje, reusa su redacción (la trampa anterior). Si las redacta el *generador*, el conjunto de evaluación mide si el sistema está de acuerdo consigo mismo. Un ítem redactado por un modelo es una propuesta hasta que una persona verificó la respuesta contra el corpus, y en el informe decís cuáles son esos ítems.

Trampa: una clave de respuesta no puede decir que no. La *contención* da por buena una respuesta si contiene *cualquier* cadena de la clave. En "¿puedo usar X en 3.12?" la clave natural es la versión en la que llegó X, "3.13", y la contienen tanto "No, X llegó en 3.13" (correcta) como "Sí, X está desde 3.13" (incorrecta). Lo mismo pasa con cada "¿puedo usar X en mi versión?", cada "¿soporta…?" y cada sonda de antes/después de

una actualización: puntúan igual con el veredicto correcto que con el incorrecto, y una memoria que nunca registró la actualización puntúa tan bien como una que sí.

Qué hacer: a esos ítems dales un `expected_verdict` (`"yes"` / `"no"`) y puntuá el veredicto aparte de la clave.

- **Leé solo la primera palabra** de la respuesta, de forma estricta. Si no es *sí* ni *no*, el veredicto es `unclear` (poco claro), y se cuenta como tal.
- **Leela sobre el texto crudo.** La normalización que saca los acentos para la contención convierte *sí* en *si*, el condicional.
- **Una abstención no es un "no".** "No lo sé" también empieza con *no*: si no lo separás, cada abstención puntúa bien en los ítems "no".
- **Una actualización solo se ve en un par.** Un brazo que dice "sí" a todo acierta cada ítem "sí". Para medir una actualización, hacé la misma pregunta antes y después de ella: veredictos opuestos, los dos correctos.

Preguntas para el informe.

- ¿Cómo evitaste que las preguntas copiaran al corpus, y qué dice tu chequeo de contaminación?
- ¿Qué fracción de tus etiquetas gold son conjuntos en lugar de ids únicos, y por qué?
- ¿Qué habilidades de memoria cubren tus escenarios, con cuántas sondas cada una, y cuáles dejaste afuera?
- Describí una pregunta que haría un usuario real y que tu conjunto no puede medir.
- Si más adelante tus columnas `topical` e `identifier` no coinciden sobre una técnica, ¿en cuál vas a confiar? ¿Y por qué conviene decidirlo ahora?

Etapa 3 · Recuperación sobre tu corpus

Para qué. Es el protocolo que usamos para RAG en el curso -- congelar la vara, mover una variable, leer la tabla --, ahora sobre un corpus que limpiaste vos. Los resultados no se transfieren entre corpus: en el corpus del curso no todas las piezas pagaron lo que costaban, y una mejora resultó venir de algo ajeno al mecanismo propio de la técnica. Averiguá qué piezas se ganan su costo *acá*.

> **No te quedes acá.** Una grilla completa de la Etapa 3 y nada después es el recorte que más discutiríamos (§2.2). Tres filas bien acotadas y una Etapa 5 real hacen un proyecto mejor.

Entra / sale. Entra: `documents.jsonl` y el conjunto congelado. Sale: `artifacts/<profile>/index/` (chunks, BM25 y el manifiesto del índice), `artifacts/<profile>/embeddings/`, y resultados por ítem en `artifacts/<profile>/results/`. Va al informe: la tabla de filas, con cotas de ruido.

Pasos sugeridos.

1. **Construí el índice.**
    - **Hijos** (los chunks que se indexan y se buscan): cortados siguiendo la estructura, y dimensionados en tokens **del reranker**, con el prefijo incluido.
    - **Padres** (lo que se le puede entregar al generador cuando un hijo gana): la unidad estructural, con un tope en tokens del generador.
    - **Prefijo contextual** a partir de metadatos: título, sección, versión, y el *año* para cualquier cosa periódica.

    - **Canal denso:** `BAAI/bge-m3`. **Canal léxico:** `bm25s` con un stemmer en español, stopwords y la normalización de identificadores **de tu corpus**.
    - **Un manifiesto del índice:** id y revisión del embedder, largo máximo de secuencia, parámetros del chunker, hash del corpus.
    - `store.cached_matrix(..., art_dir=ART)` guarda los embeddings en `ART/embeddings/`. La clave de la caché es la especificación del encoder que le pasás más los ids de los chunks *y sus textos*: un prefijo nuevo o un re-corte genera una matriz nueva, nunca reusa una vieja.
2. **Una función de recuperación, un flag por pieza.** Si un flag pide una pieza que no está cargada, la función **lanza una excepción** (ver el fragmento). Cada resultado registra la configuración que lo produjo.
3. **Las filas.** Dos líneas de base: **solo BM25** (que además sirve como detector de contaminación) y **solo denso**. A partir del denso, cada fila agrega una pieza a la anterior: + prefijo → + léxico vía RRF → + rerank → + padre → + filtro de metadatos (acá, parseado de la pregunta; en la Etapa 6, tomado de la memoria). HyDE, si lo probás, es una *rama etiquetada* aparte, no una fila acumulativa.
4. **Métricas, por categoría:**
    - hit@k y MRR a nivel de documento;
    - hit@k a nivel de **sección**;
    - *answer-delivered* (respuesta entregada): la clave de respuesta aparece en el texto que se le entregó al generador;
    - tokens del generador entregados;
    - latencia;
    - *opcional*, para las preguntas fuera del corpus: la distribución de los puntajes máximos. ¿Puede la recuperación sola decirte que la respuesta no está?
5. **Entregá por presupuesto de tokens, no por k** (ver la trampa "el mismo k").
6. **Acotá el ruido**, para cada fila contra la anterior:
    - victorias / derrotas / empates pareados;
    - un test de signos o un test exacto de McNemar;
    - un intervalo bootstrap por fila.
7. **Abrí la caja** en el movimiento más grande: mirá los rankings de cada canal para las preguntas que cambiaron de resultado.
8. **Chequeá la ventana del reranker.** El reranker lee `max_length` tokens de (consulta + hijo) y descarta el resto en silencio. Imprimí la proporción de tus pares que supera ese límite, contada en el tokenizer del propio reranker.

```
from dataclasses import asdict, dataclass

@dataclass(frozen=True)
class RetrievalConfig:
    dense: bool = True
    prefix: bool = False
    lexical: bool = False
    rerank: bool = False
    parent: bool = False
    metadata_filter: bool = False
    budget_tokens: int = 3000   # entregados al generador, contados en SU
tokenizer

def retrieve(query: str, cfg: RetrievalConfig, index) -> list[dict]:
```

```
    # Lanzá una excepción, no la saltees. Una pieza que avisa y sigue produce una
fila idéntica
    # a la anterior, y vas a leer "sin efecto" en una tabla que nunca corrió la
pieza.
    if cfg.prefix and index.prefixed_matrix is None:
        raise RuntimeError("prefix=True but no embedding matrix of prefixed chunks
is loaded")
    if cfg.lexical and index.bm25 is None:
        raise RuntimeError("lexical=True but no BM25 index is loaded")
    if cfg.rerank and index.reranker is None:
        raise RuntimeError("rerank=True but no reranker is loaded")
    if cfg.parent and index.parents is None:
        raise RuntimeError("parent=True but no child -> parent map is loaded")
    if cfg.metadata_filter and index.metadata is None:
        raise RuntimeError("metadata_filter=True but no document metadata is
loaded")
    ...

def rrf(ranked_lists: list[list[str]], k: int = 60) -> list[str]:
    scores: dict[str, float] = {}
    for ranking in ranked_lists:
        for rank, chunk_id in enumerate(ranking, start=1):
            scores[chunk_id] = scores.get(chunk_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

def diff_rows(prev: RetrievalConfig, cur: RetrievalConfig) -> str:
    changed = [k for k, v in asdict(cur).items() if asdict(prev)[k] != v]
    if not changed:
        return "identical configuration -- a duplicate row, not a new one"
    return (f"one variable: {changed[0]}" if len(changed) == 1
            else f"{len(changed)} variables differ ({', '.join(changed)}) -- NOT a
cumulative row")
```

Trampa: un acierto de documento no es una respuesta entregada. El hit@k a nivel de documento cuenta un acierto cuando llega *cualquier* pasaje del documento gold: por ejemplo, el encabezado y el primer artículo, mientras la respuesta está diez artículos más adelante. Por construcción, tampoco puede ver el efecto de entregar el padre, porque el padre siempre está en el mismo documento que el hijo. Resultado: la recuperación parece cerca de su techo, y las respuestas "no está en el contexto" de la Etapa 4 se le atribuyen a la generación.

Qué hacer: etiquetá el gold a nivel de sección, chequeá si la clave de respuesta fue *entregada*, reportá las dos cosas, y leé la brecha entre ellas antes de empezar la Etapa 4.

Trampa: "el mismo k" no es el mismo presupuesto. Con `k = 5`, los padres pueden entregar el doble de tokens que los hijos, y más tokens aumentan la probabilidad de que la cadena de la respuesta esté en algún lugar del contexto. La mejora se le acredita a la técnica cuando vino del volumen. Es el mismo error que comparar dos formas de modelo que difieren en tamaño y acreditarle la diferencia a la forma: un presupuesto comparable es una restricción que imponés vos, no viene dado.

Qué hacer: llená cada fila hasta el mismo presupuesto de tokens (en tokens del generador), reportá los tokens realmente entregados por fila, e imprimí una advertencia cuando dos filas difieran en más de una tolerancia

que declares.

Trampa: RRF supone que los canales fallan de forma independiente. Cuando una consulta engaña a los dos canales de la misma manera (el denso cae en un grupo de pasajes parecidos, BM25 en los mismos tokens genéricos), su acuerdo rankea *más alto* los pasajes equivocados, y el consenso entierra el correcto. Antes de acreditarle (o culpar) a una pieza, mirá los rankings por canal de las preguntas que hizo cambiar.

Calculá tu piso de ruido antes de afirmar nada.

- Con 30 resultados binarios cerca del 85%, el intervalo del 95% de una sola fila abarca aproximadamente ±12 puntos.
- En una comparación *pareada*, solo las preguntas discordantes (las que cambiaron de resultado entre las dos filas) llevan información. Si `n` preguntas cambiaron y todas se movieron para el mismo lado, el test de signos bilateral da `p = 2 × 0.5^n`: un 5-0 da alrededor de 0,06; un 6-0, alrededor de 0,03.
- Así que "26/30 → 28/30" no es evidencia de nada. Un resultado débil que podés acotar vale más que uno fuerte que no podés.

Preguntas para el informe.

- ¿Qué piezas se ganaron su costo en tu corpus, por encima de tu cota de ruido, y cuáles sacaste?
- Para el movimiento más grande, ¿qué mostró abrir la caja? ¿Ganó por la razón que da su paper?
- ¿Qué tan separados están tus números a nivel de documento y a nivel de sección (o de respuesta entregada), y qué predice eso para la Etapa 4?
- Con tokens igualados, ¿es mejor entregar muchos hijos o pocos padres?
- ¿Ayudó el filtro de metadatos, y en qué categoría?

Etapa 4 · Respuestas fundamentadas con un presupuesto en el que podés confiar

Para qué. La recuperación fija el techo; la generación decide cuánto de ese techo se alcanza. Esta etapa mide esa segunda superficie, y construye el componente del que dependen todas las etapas posteriores: **un ensamblador de prompts que llena un presupuesto de tokens, con una cuenta que verificaste contra la del propio servidor.**

Entra / sale. Entra: cualquier configuración de recuperación de la Etapa 3 y el conjunto congelado. Sale: el ensamblador (lo reusan las Etapas 5 y 6), resultados por ítem, y la caché del LLM en `artifacts/<profile>/llm_cache/`. Va al informe: las tres filas y la tabla 2×2.

Antes de empezar, leé §6.2: cómo trata Ollama tu prompt. En corto: la API no tiene estado, `num_ctx` se manda en cada llamada (si no, en una T4 vale 4.096), y un prompt que no entra no da error, se trunca.

Pasos sugeridos.

1. **El ensamblador.** Arma el prompt: sistema + pasajes delimitados (marcadores `[doc_id · section]`) + pregunta, dejando una reserva de salida igual a `num_predict`.
    - El prompt entero se cuenta con la plantilla de chat del generador: `llm.count_prompt_tokens(messages, model)`.
    - Para contar cada pasaje (el `count` que reciben los fragmentos de abajo) alcanza con `lambda t: llm.count_tokens(t, MODEL)`. Ojo: cuenta texto pelado y no ve los marcadores de plantilla de cada mensaje. Dejá un margen, y dejá que `llm.chat` verifique el total.

- Si un pasaje no entra, saltealo y seguí con el siguiente; no cortes el loop (los que vienen pueden ser más chicos).
- Cada llamada registra tu cuenta (`r.expected_prompt_tokens`) *y* el `prompt_eval_count` del servidor (`r.prompt_tokens`).

2. **La generación.** `qwen3:8b`, `think=False`, temperatura 0, un schema de Pydantic como `format` (`llm.structured`), y un `num_ctx` explícito en cada llamada, a través de la API nativa.

   - **El campo de razonamiento va primero.** Declaralo primero, mantenelo obligatorio, y dale un nombre que también quede primero alfabéticamente. Según la versión del servidor, las claves del JSON se escriben en el orden del schema (la versión que fija este lab hace eso) o en orden alfabético. En los dos casos querés que el modelo razone *antes* de comprometerse con una respuesta. `llm.probe_key_order` te dice qué hace tu servidor.

3. **Puntuá sin juez:**

   - la abstención, **a partir del booleano del schema** (`can_answer`), nunca buscando texto;
   - citas ⊆ ids entregados (¿citó solo lo que se le dio?);
   - citados ∩ gold (¿citó lo correcto?);
   - contención de la clave de respuesta, después de normalizar acentos, formatos de números y comas decimales;
   - el veredicto de los ítems de sí/no, aparte de la clave (Etapa 2);
   - flags de consistencia: el booleano dice "no hay suficiente contexto" pero la respuesta no es una abstención, o viceversa.

   Una llamada con `r.error` seteado (una respuesta que no se pudo parsear, un prompt truncado, un rechazo del servidor) es su **propia categoría**: nunca una abstención, nunca una respuesta puntuada. Lo mismo vale para un `llm.ContextOverflow`, que no llega como `r.error` sino como excepción: atrapala y registrala en esa categoría.

4. **Tres filas sobre las mismas preguntas:**

   - **libro cerrado** (sin recuperación);
   - tu **mejor configuración** de la Etapa 3;
   - **oráculo** (se le entrega directamente la sección gold).

5. **La tabla 2×2, por pregunta:** recuperó bien / mal × respondió bien / mal. Sirve para separar las dos fallas que importan:

   - *fiel al pasaje equivocado*: recuperó mal, y respondió lo que decía ese pasaje;
   - *lo tenía, respondió otra cosa*: recuperó bien, y respondió mal.

6. ***Opcional:* posición del pasaje.** El pasaje con la respuesta primero, en el medio o último, con una cantidad fija de pasajes y de tokens. Rellená hasta 10-20 pasajes si querés tener chances de ver un efecto.

7. **Mirá saltar la guarda una vez, a propósito** (ver "Las guardas de `llm.chat`", abajo).

```
from pydantic import BaseModel, Field
import llm
```

```python
class GroundedAnswer(BaseModel):
    # Declarados en el orden en que el modelo debería escribirlos, obligatorios, y
con nombres
    # tales que el orden alfabético coincida ("analysis" < "answer" < "can_answer"
< "citations"):
    # el razonamiento viene ANTES que la respuesta en cualquier servidor.
    analysis: str = Field(max_length=600)       # un tope, para que un análisis
largo no se coma num_predict
    answer: str = Field(max_length=1200)
    can_answer: bool
    citations: list[str]

def assemble(passages: list[dict], budget: int, count) -> tuple[list[dict],
list[dict]]:
    delivered, skipped, used = [], [], 0
    for p in passages:
        n = count(p["text"])
        if used + n > budget:
            skipped.append(p)   # saltear y registrar -- no `break`: un break en
el primer pasaje
            continue            # que no entra descarta también todos los más
chicos que vienen después
        delivered.append(p)
        used += n
    return delivered, skipped

answer, r = llm.structured(messages, GroundedAnswer, model="qwen3:8b",
num_ctx=8192, num_predict=700)
print(r.expected_prompt_tokens, r.prompt_tokens, r.done_reason, r.error)
```

Las guardas de `llm.chat`.

- **Antes de mandar:** si tu prompt + `num_predict` supera `num_ctx`, `llm.chat` lanza la excepción `ContextOverflow` y no manda nada.
- **En el servidor:** con `strict_ctx=True` (el valor por defecto), `llm.chat` le pide al servidor que rechace en lugar de truncar. Un rechazo vuelve en `r.error`, como `CONTEXT_OVERFLOW (server)`.
- **Después:** `SILENT_TRUNCATION`, en `r.error`, marca un prompt que el servidor evaluó solo en parte.
- El chequeo de antes de mandar y `SILENT_TRUNCATION` necesitan el tokenizer del modelo. Funcionan para los tags de `llm.HF_TOKENIZERS`; para cualquier otro tag, primero agregá ahí su repo de HuggingFace.
- **Ejercicio:** apagá la guarda una vez (`strict_ctx=False`), a propósito, con un prompt demasiado grande, y mirá qué vuelve.

Trampa: el servidor le corta el frente a un prompt demasiado grande. Cuando un único prompt supera `num_ctx`, Ollama conserva apenas los primeros tokens y la *cola*, y descarta todo lo del medio. Lo que se va son tus instrucciones, la regla de abstención y los pasajes mejor rankeados; la pregunta del final sobrevive. El modelo igual responde con fluidez (a partir de tus pasajes peor rankeados), y `format` igual hace que la salida sea JSON válido. El servidor lo registra una sola vez, como una advertencia en un log que nadie lee.

Es fácil chocarlo: el valor por defecto en una T4 es 4.096 tokens, así que cinco pasajes y un prompt de sistema alcanzan. Y el endpoint `/v1` compatible con OpenAI no puede fijar `num_ctx` por llamada, así que cualquier framework apuntado a él hereda el valor por defecto del servidor.

Las conclusiones equivocadas que produce: "el reranker no ayuda a la generación" (sus mejores pasajes fueron los primeros en irse), "el modelo ignora la regla de abstención", "más pasajes perjudican".

Qué hacer: fijá `num_ctx` en cada llamada, mantené el prompt por debajo de `num_ctx − num_predict`, y compará `prompt_eval_count` con tu propia cuenta en cada llamada.

Trampa: buscar la abstención en el texto. Contá lo que te da el schema. La abstención es un booleano: buscar la cadena "No lo sé" la cuenta mal, porque el modelo puede parafrasear, o escribir "No lo sé" en el análisis mientras responde en el campo de respuesta. Y una cita es una *afirmación* del modelo: verificala contra lo que realmente se entregó.

Trampa: buscar un efecto de posición en una ventana chica. Los efectos de posición necesitan una ventana en la que valga la pena perderse. Con cinco pasajes en cinco mil tokens, un resultado nulo no refuta *Lost in the Middle*. Si evaluás la posición, decí qué tamaño de prueba *podría* haber mostrado el efecto.

Preguntas para el informe.

- De las preguntas en las que llegó el pasaje con la respuesta, ¿cuántas se respondieron bien, y cómo eran las fallas?
- ¿Qué tan grande es la brecha entre la fila oráculo y tu mejor fila de recuperación? ¿Y entre libro cerrado y tu mejor fila?
- ¿Con qué frecuencia el modelo citó un pasaje que no se le dio, o se abstuvo mientras la respuesta estaba en su contexto?
- Mostrá un prompt demasiado grande: tu cuenta, la cuenta del servidor, y qué respondió el modelo.
- ¿Importó la posición a tu escala, y podría tu prueba haberlo mostrado si importara?

Etapa 5 · La conversación: una ventana, dos fuentes *(el corazón del lab)*

Para qué. La API no tiene estado: todo lo que el modelo "recuerda" dentro de una sesión es lo que tu código vuelve a poner en la ventana en cada turno. Una vez que hay una conversación, dos cosas compiten por la ventana -- los pasajes del corpus y el historial --, y cada turno tiene las mismas dos superficies de falla que el RAG de un solo turno:

- **con qué busca el recuperador** (Parte A);
- **qué ve el generador** (Parte B).

Entra / sale. Entra: los guiones de la Etapa 2, tu recuperador y el ensamblador de la Etapa 4. Sale: resultados por turno. Va al informe: la tabla de la Parte A por dependencia y por turno, y el experimento de la Parte B.

Parte A · Con qué buscar

Pasos sugeridos.

1. **Compará cuatro consultas** sobre tus guiones, contra el gold de cada turno:
    - solo el **último turno** del usuario;

- la **conversación completa**;
- una **reescritura condensada** del último turno, hecha por un LLM a partir del historial reciente (con un schema de un solo campo, por ejemplo `{"query": str}`);
- tu reescritura **oráculo**, escrita a mano (`oracle_rewrite`, Etapa 2).

Acordate de Ye et al. (Apéndice B.3): la reescritura humana no siempre es la mejor consulta.

2. **Reportá** hit@k y hit@k de sección **por etiqueta de dependencia y por número de turno**.

3. **Leé 20 reescrituras a mano.** Un reescritor que responde la pregunta, agrega detalles específicos o incluye sus pasos de razonamiento dirige la recuperación como lo hace HyDE: por razones ajenas a reescribir.

Parte B · Qué conservar (el experimento central)

Pasos sugeridos.

4. **Implementá estrategias de historial:**

   - el **buffer completo**, solo mientras realmente entre (chequealo);
   - una **ventana** de los últimos N turnos;
   - un **buffer de resumen** al estilo MemGPT: un resumen continuo más los turnos recientes, que se vuelve a resumir cuando el historial cruza un umbral de su presupuesto;
   - *opcional:* compresión extractiva de pasajes (conservar las oraciones que tu reranker puntúa más alto) para hacer lugar.

5. **El experimento.**

   - Fijá `num_ctx` (por ejemplo, 8.192) y un presupuesto total de prompt para pasajes + historial.
   - Mové **una sola** cosa: la división entre pasajes e historial (por ejemplo, 80/20 vs 50/50 vs 30/70 del total), *o* una elección de compresión con la división fija.
   - Todo lo demás queda fijo: los guiones, el recuperador, la consulta ganadora de la Parte A, la reserva de salida.

6. **Hacé que el historial sea una restricción real.** Un guion de 3-6 turnos son unos pocos cientos de tokens de historial. Frente a un presupuesto de historial de mil o más, todas las estrategias conservan todo, la división solo mueve el presupuesto de pasajes, y la tabla dice "el historial no importa". Para evitarlo, hacé una de estas dos cosas, o las dos:

   - reproducí tus guiones uno detrás de otro como una sola conversación larga (cada turno conserva su propio gold y el id de su guion, para el bootstrap);
   - imponé un total lo bastante chico como para que el lado del historial de cada división restrinja en buena parte de los turnos.

   Antes de la primera respuesta, imprimí los tokens de historial por turno contra el presupuesto de cada división. (§7.3 sugiere números.)

7. **Reproducí historiales guionados.** Cada brazo ve el *mismo* historial -- las respuestas guionadas del asistente de la Etapa 2 -- y solo se genera el turno actual. Si cada brazo se realimentara con sus propias respuestas anteriores, los brazos divergirían desde el turno 2 y cada comparación posterior quedaría confundida. Además, reproducir cuesta una fracción de las llamadas.

8. **Métricas por turno:** contención de la respuesta, abstención correcta en los turnos sin respuesta, tokens y llamadas al LLM por turno, latencia. Acotá el ruido **por guion**, no por turno (ver la última trampa).

9. **Poné a prueba el resumen.** Al final de cada guion, preguntá por un detalle de los turnos 1-2, y auditá el texto del resumen buscando hechos inventados, relaciones equivocadas y detalles faltantes (las tres categorías de Wang et al., Apéndice B.6).

```
def turn_messages(system: str, history: list[dict], passages: list[dict],
question: str,
                  hist_budget: int, pass_budget: int, count) -> list[dict]:
    # Armá la ventana vos, en cada turno, dentro de un presupuesto que
verificaste. No le pases al
    # servidor una lista de mensajes que crece confiando en que entre: pasado
num_ctx descarta los
    # turnos más viejos sin avisarte (ver la Trampa de abajo).
    kept, used = [], 0
    for msg in reversed(history):                # los más nuevos primero
        n = count(msg["content"])
        if used + n > hist_budget:
            break                                # acá descartar los turnos MÁS
VIEJOS es justamente la idea
        kept.insert(0, msg)
        used += n
    if kept and kept[0]["role"] == "assistant":   # no arranques con una respuesta
sin su pregunta
        kept.pop(0)
    context, _ = assemble(passages, pass_budget, count)
    context_block = "\n\n".join(f"[{p['doc_id']} · {p['section']}]\n{p['text']}"
for p in context)
    return ([{"role": "system", "content": system}] + kept +
            [{"role": "user", "content": f"
<contexto>\n{context_block}\n</contexto>\n\n{question}"}])

class Rewrite(BaseModel):
    query: str = Field(max_length=300)   # un solo campo: nada con qué razonar,
nada que responder

def conversation_query(history: list[dict], question: str) -> str:
    # Para la configuración de conversación completa: el turno MÁS NUEVO primero.
El embedder
    # trunca por la derecha en su max_seq_length, y una concatenación en orden
cronológico pierde
    # justamente el turno que dice qué se está preguntando ahora. Solo los turnos
del usuario --
    # una elección: incluí las respuestas del asistente si tu "conversación
completa" significa
    # las dos cosas, y decí cuál hiciste.
    return " \n".join([question] + [m["content"] for m in reversed(history) if
m["role"] == "user"])
```

Trampa: tu brazo de "historial completo" es en secreto una ventana deslizante. Cuando los mensajes de una llamada a `/api/chat` superan `num_ctx`, Ollama **descarta los mensajes más viejos** (conserva el mensaje de sistema y los más recientes), y lo registra solo en nivel debug. Consecuencias:

- pasada la ventana, tu "buffer completo" es en secreto "prompt de sistema + los últimos turnos", y la comparación "completo vs. ventana vs. resumen" queda confundida;
- "el modelo se olvidó de lo que dije en el turno 2" es el servidor, no el modelo;
- el brazo con "más historial" en realidad no recibió más.

Qué hacer: que tu ensamblador decida qué entra (como `turn_messages`), y que `prompt_eval_count` lo confirme.

Trampa: una consulta con la conversación completa pierde su último turno. Los embedders de oraciones truncan por la derecha en el `max_seq_length` que fijes (§7.1 sugiere 512-1.024). Una conversación concatenada en orden cronológico conserva sus *primeros* turnos y descarta en silencio el más nuevo, que es el único que dice qué se está preguntando ahora. El resultado coincide con un hallazgo publicado (MTRAG: la conversación completa rinde menos que el último turno; Apéndice B.3), pero por una razón que no tiene nada que ver con ese hallazgo, y empeora turno a turno a medida que crece la conversación. Un único guion de 3-6 turnos puede no llegar nunca al límite (contá la consulta en tokens del embedder); tus guiones reproducidos uno detrás de otro sí van a llegar.

Qué hacer: poné el último turno primero (como `conversation_query`), o truncá por la izquierda, contando en el tokenizer del propio embedder. BM25 no tiene ese límite, así que ahora los dos canales ven consultas distintas: vale una oración en tu informe.

Trampa: comparar brazos con totales distintos. Un total igualado es una restricción que imponés vos, en tu ensamblador. Compará los brazos con el mismo total de tokens de prompt *tal como los contó el servidor*. Y mantené `num_ctx` constante entre brazos: cada cambio de `num_ctx` vuelve a cargar el modelo, así que intercalar brazos que difieren en él convierte tu columna de latencia en un conteo de recargas.

Trampa: guardar el razonamiento en el historial. Lo que entra al historial es lo que vio el usuario: guardá la `answer` (y sus citas), no el campo `analysis`. La propia recomendación de Qwen es no conservar el razonamiento en el historial, y el campo de análisis *es* razonamiento.

Trampa: tratar los turnos como independientes. Cincuenta turnos de diez guiones no son cincuenta muestras. Al hacer bootstrap, remuestreá guiones, no turnos.

Preguntas para el informe.

- ¿Qué configuración de consulta ganó en qué tipo de dependencia, y cambia el ranking con el número de turno?
- Con una ventana fija, ¿dónde quedó la mejor división pasajes/historial para tu corpus, y la diferencia está por encima de tu cota de ruido?
- ¿Qué hicieron mal tus resúmenes, contado en las tres categorías de la auditoría (hechos inventados, relaciones equivocadas, detalles faltantes)?
- ¿Cuánto cuesta cada estrategia por turno, en llamadas y tokens, y vale la pena el ganador?
- ¿Dónde perdió la reescritura oráculo frente a la del modelo, o le ganó?

Etapa 6 · Memoria entre sesiones

Para qué. Entre sesiones no sobrevive nada que no escribas. Acá la memoria es un corpus que tu sistema escribe *en tiempo de ejecución*, a partir de una fuente que puede estar equivocada (su propia extracción), y que tiene que mantener actualizado.

En el vocabulario de CoALA (Apéndice B.4), que vas a usar en tu informe:

- la ventana ensamblada es la memoria **de trabajo**;
- las transcripciones y los resúmenes de las sesiones son memoria **episódica**;
- los hechos extraídos son memoria **semántica**;
- las preferencias sobre *cómo* responder son memoria **procedimental**.

La línea de base honesta es "reproducir todo lo que entre", y en la evaluación propia de Mem0 esa línea de base ganó en exactitud (Apéndice B.4).

Entra / sale. Entra: los escenarios de la Etapa 2 y el loop de la Etapa 5. Sale: `artifacts/<profile>/memory/` (el almacén de memoria y su log de operaciones) y resultados por sonda. Va al informe: los resultados por habilidad.

Pasos sugeridos.

1. **Líneas de base:** sin memoria (el piso), y reproducción completa de las sesiones pasadas, todo lo que entre. **Declará el régimen:** ¿el historial de tu escenario entra en la ventana, o no?

2. **El camino de escritura.** Después de cada sesión (o de cada turno):

   - extraé hechos candidatos con un schema;
   - recuperá los recuerdos existentes más parecidos;
   - decidí **ADD / UPDATE / NOOP / INVALIDATE**;
   - guardá `created_at`, `valid_from` y `valid_to`.

   Invalidá, no borres: "¿qué había dicho antes?" tiene que seguir teniendo respuesta. (Es la idea de Zep/Graphiti, Apéndice B.4. No necesitás un grafo: alcanzan unas columnas de timestamp.)

3. **El camino de lectura.** Recuperá recuerdos por relevancia (con el mismo embedder), opcionalmente sumando recencia e importancia al estilo de Generative Agents, siempre sobre los timestamps **simulados** del escenario. Ponelos en un **espacio de memoria** (*memory slot*) del ensamblador de la Etapa 4, dentro de su presupuesto.

4. **La memoria alimenta la recuperación.** Los hechos del usuario que se corresponden con metadatos del corpus se convierten en filtros: la versión que usa, la fecha "a la cual" pregunta, la región que sigue (corpora.md propone un gancho de memoria por corpus). Medilo en las sondas que lo necesitan, y notá cuando el filtro *saca* justo el pasaje que le permitiría al modelo decir "eso no está disponible para vos".

5. **Métricas:**

   - exactitud de las sondas por habilidad, reportada como **conteos** (3/4, no 75%);
   - recall de las sesiones con evidencia (sin juez: las sesiones están etiquetadas en `evidence_sessions`);
   - tokens y llamadas al LLM por sesión;
   - cuántos registros guardados están mal, o registran algo que nunca se dijo.

6. **Compará** sin memoria / reproducción / almacén, con presupuestos declarados. El resultado de Mem0 predice que el almacén gana en costo y pierde en exactitud *donde la reproducción entra*. Reproducilo o refutalo a tu escala, y encontrá dónde la reproducción deja de entrar.

```
from datetime import datetime
from typing import Literal

class MemoryRecord(BaseModel):
    memory_id: str
    kind: Literal["semantic", "episodic", "procedural"]
    subject: str                      # por ejemplo, "python_version" -- un único
valor ACTUAL por subject
    text: str
    created_at: datetime              # cuándo lo escribió el sistema
    valid_from: datetime              # cuándo pasó a ser cierto (el tiempo
simulado de la sesión)
    valid_to: datetime | None = None  # se setea en INVALIDATE; el registro se
queda
    source_session: str
    importance: int = Field(default=5, ge=1, le=10)

class WriteDecision(BaseModel):
    # El razonamiento declarado primero, obligatorio, y con un nombre que también
quede primero
    # ("analysis" < "decision" < "target_id" < "text"). Si la decisión se llamara
"action", en un
    # servidor que escribe las claves en orden alfabético el modelo decidiría
primero y
    # justificaría después.
    analysis: str = Field(max_length=400)
    decision: Literal["ADD", "UPDATE", "NOOP", "INVALIDATE"]
    target_id: str | None = None
    text: str | None = None

def score(recency: list[float], importance: list[float], relevance: list[float]) -
> list[float]:
    def minmax(xs):
        if not xs:
            return []
        lo, hi = min(xs), max(xs)
        if hi - lo < 1e-6:   # Solo si es EXACTAMENTE constante. Un término casi
constante (la recencia
            return [0.0] * len(xs)   # de reloj en una reproducción) igual se
estira: imprimí antes cada rango.
        return [(x - lo) / (hi - lo) for x in xs]
    return [a + b + c for a, b, c in zip(minmax(recency), minmax(importance),
minmax(relevance))]
```

Trampa: guardar hipótesis como hechos. Una respuesta estructurada guardada entera lleva consigo su campo `analysis`. Un escritor de memoria o un resumidor que conserva ese texto guarda **hipótesis**

formuladas como hechos ("el usuario probablemente usa 3.11"), que después recupera como cosas que dijo el usuario.

Qué hacer: guardá solo los campos del schema que querés conservar (usá un schema también para los resúmenes), chequeá que ningún registro guardado contenga un análisis o un bloque `<think>`, y reportá el conteo.

Trampa: el razonamiento se cuela por el `content`. Se reportó que qwen3 filtra `</think>` en el `content` incluso con `think=False`, y la forma en que las bibliotecas cliente y las versiones del servidor separan `thinking` de `content` fue cambiando con el tiempo. `llm.chat` saca esas filtraciones y las *cuenta*: ese conteo es una línea de tu informe.

Un costo relacionado, sin ninguna filtración: un framework que no manda `think` deja que el servidor *active* el thinking para qwen3, y cada llamada gasta en silencio tokens de salida en un razonamiento que después se tira.

Trampa: normalizar un término casi constante. Chequeá el rango antes de normalizar. En una reproducción guionada el reloj real casi no se mueve, así que una recencia medida con el reloj es casi constante, y min-max estira ese ruido a [0, 1], donde pesa tanto como la relevancia. Usá los timestamps simulados del escenario, e imprimí el rango de cada término.

Declará el régimen. Cada afirmación de "la memoria le gana / pierde contra el contexto completo" dice si el historial entraba en la ventana, y con qué `num_ctx`.

Preguntas para el informe.

- ¿En qué habilidades la memoria le ganó a no tener memoria, y en cuáles la reproducción le ganó a la memoria? ¿Con qué costo en tokens?
- Mostrá una actualización de conocimiento de punta a punta: qué se guardó, qué se invalidó, qué obtuvo la sonda.
- ¿Qué guardó tu extractor que no era cierto, o que no se dijo?
- ¿Alguna memoria del usuario mejoró la recuperación como filtro, o la perjudicó?
- ¿En qué punto tu escenario deja de entrar en la ventana, y qué cambia entonces?

Etapa 7 · Una segunda lectura: juez y framework *(opcional, recomendada)*

Para qué. Las dos partes son segundas lecturas de lo que construiste: el juez (7a) lee las respuestas; el framework (7b) lee el diseño.

Entra / sale. Entra: cualquier conjunto de respuestas guardadas (7a), o tu loop de las Etapas 5/6 (7b). Sale: los veredictos del juez y su auditoría (7a); resultados por sonda con el framework (7b). Va al informe: la auditoría del juez, si la hiciste, y la comparación con el framework.

7a · Un juez

Para respuestas abiertas que la contención no puede puntuar.

- **Qué recibe el juez:** la pregunta, la clave de respuesta y la respuesta, como **campos separados y etiquetados**. Incluí el contexto recuperado *solo* cuando evalúes la fidelidad a ese contexto; si no, premiás ser fiel al pasaje equivocado.

- **Una rúbrica por tipo de ítem.** Las reglas de LongMemEval para actualizaciones de conocimiento y abstención son un buen punto de partida (Apéndice B.5).
- **Un veredicto estructurado, temperatura 0.**
- **Auditá a mano al menos 30 juicios o el 20%, lo que sea mayor**, y reportá la matriz de confusión del juez contra la contención y contra vos.
- **Autopreferencia.** Si el juez es el mismo modelo que el generador, nombrala. Si podés, corré un juez de otra familia de modelos, en una pasada separada que lea tus respuestas guardadas (un modelo de 8B a la vez entra cómodo en una T4). Agregá su tokenizer a `llm.HF_TOKENIZERS`, o los chequeos de contexto de `llm.chat` quedan desactivados para él.

Un juez es un instrumento: calibralo antes de leerlo. El puntaje promedio de un juez sin calibrar es un número sobre el juez.

7b · Un framework

Reimplementá tu loop de las Etapas 5/6 en uno de estos:

- LangGraph (un checkpointer para la memoria del hilo, un store para la memoria entre hilos);
- la `Memory` de LlamaIndex, con memory blocks;
- el middleware de resumen de LangChain 1.x (no el deprecado `ConversationBufferMemory`).

Mapealo pieza por pieza con el tuyo, volvé a correr tus sondas, y compará tokens y respuestas. Después nombrá **qué decidió el framework por vos**: cuándo resume, qué guarda, su ventana por defecto, si manda `think` o no, y si por defecto usó un modelo pago o el endpoint `/v1` (y por lo tanto la ventana por defecto del servidor, 4.096 tokens en una T4).

Para instalarlo: `requirements-extras.txt` tiene versiones que se resuelven junto con las del núcleo. Copiá las líneas que uses a `requirements-group.txt` (Puesta en marcha, punto 5).

Preguntas para el informe.

- ¿Dónde el juez no coincidió con la contención y con tu auditoría, y en cuál confiás?
- ¿Cuánto importa acá la autopreferencia, y cómo lo sabés?
- ¿Qué decidió el framework por vos que vos habías decidido a mano, y cambió tu presupuesto o tus resultados?

---

4 · Artefactos y traspasos

Cada etapa consume la salida de las anteriores, así que acuerden los nombres de archivo antes de que alguien empiece, sobre todo si reparten las etapas entre integrantes. Los scripts de las etapas se llaman `01_documents.py`, `02_eval_set.py`, `03_retrieval.py`, `04_grounded_answers.py`, `05_conversation.py`, `06_memory.py` y `07_second_read.py` (Puesta en marcha, punto 4).

- **Commiteá** lo que es chico y hecho a mano. **Regenerá** lo que es grande, y verificalo al cargar.
- Los artefactos generados viven en `artifacts/smoke/` o `artifacts/full/` (`environment.artifacts_dir(SMOKE_TEST)`): así un índice smoke nunca se lee como el real. `artifacts/` está en el gitignore; en Colab puede vivir en Drive.

Artefacto	Lo escribe	Lo lee	¿En git?
`corpus_card.md`	Etapas 0-1	informe	commit
`artifacts/<profile>/documents.jsonl` + manifiesto (versión fijada de la fuente, conteos, hash del corpus)	Etapa 1	Etapas 2, 3, 6	no
`eval/single_turn.jsonl`, `eval/dialogues.jsonl`, `eval/scenarios.jsonl`	Etapa 2	Etapas 3-7	commit
`eval/FROZEN.json`	Etapa 2 (`schema.freeze_eval`)	cada etapa que calcula una métrica (`schema.load_eval`)	commit
`artifacts/<profile>/index/` -- chunks, BM25 y el manifiesto del índice	Etapa 3	Etapas 3-6	no
`artifacts/<profile>/embeddings/` (`store.cached_matrix(..., art_dir=ART)`)	Etapa 3	Etapas 3-6	no
`artifacts/<profile>/llm_cache/` (`llm.use_cache(ART / "llm_cache" / "qwen3_8b.jsonl")`)	Etapas 4-7	las mismas etapas, al volver a correr	no
`artifacts/<profile>/results/*.jsonl` -- una fila por ítem por brazo (`store.ResultsLog`)	Etapas 3-7	tablas, informe	va en el zip de la entrega
`artifacts/<profile>/memory/` -- el almacén de memoria y su log de operaciones	Etapa 6	Etapas 6-7	va en el zip
`artifacts/<profile>/run_report.json`	cada etapa (`console.StageRun`)	vos, y nosotros	va en el zip
`decision_log.md`	todas las etapas	quienes corrigen	commit

Procedencia: cada artefacto sabe de dónde salió

La forma más común de obtener una tabla equivocada es puntuar un índice nuevo contra un conjunto de evaluación viejo, o un índice smoke contra el corpus completo: ningún error, solo números sobre otra cosa. Para evitarlo:

- **Un índice** sabe qué `documents.jsonl` (por hash) y qué embedder lo construyeron.
- **Una fila de resultados** sabe su configuración y el hash del conjunto de evaluación contra el que se puntuó. Poné las dos cosas en el `run_spec` de `ResultsLog`, por ejemplo `{"config": asdict(cfg), "eval": ev.key()}`: las filas tienen como clave ese hash y nada más. (`ev.key()` son los hashes

congelados o, para el conjunto smoke sin congelar, un hash de sus filas. Así un ítem smoke editado se recalcula, nunca se reanuda.)

- **Una etapa que carga un artefacto** verifica su manifiesto y rechaza una discrepancia (`store.check_manifest`, `schema.load_eval(..., documents_path=...)`).
- **Antes de calcular una sola métrica**, verificá que cada id gold esté en el índice. `schema.validate_eval` los verifica contra los documentos; el chequeo contra tu índice lo escribís vos.

---

5 · Entregables y rúbrica

5.1 · Qué entregás

Subí al campus **un solo zip**, a más tardar el **domingo 2026-10-25**, con:

- **Notebook(s) con sus salidas**, que cubran las etapas que elegiste. Leemos las salidas: un notebook sin ellas no nos muestra nada.
- **Informe** (PDF, o markdown en un notebook), con:
  - las decisiones de diseño y su justificación;
  - la ficha del corpus (*corpus card*);
  - la descripción del conjunto de evaluación y su chequeo de contaminación;
  - la tabla de la Etapa 3, con cotas de ruido;
  - las filas de la Etapa 4 y la tabla 2×2;
  - la tabla de la Etapa 5A por dependencia y por turno, y el experimento de la Etapa 5B;
  - los resultados de la Etapa 6 por habilidad;
  - la auditoría del juez, si la hiciste;
  - las respuestas a las preguntas de las etapas que hiciste;
  - el párrafo sobre cómo usaste IA (§2.3).
- **El registro de decisiones** (`decision_log.md`, §2.4).
- **Lo necesario para recalcular tus tablas:** `eval/` con `FROZEN.json`, tu `corpus_card.md`, y de `artifacts/<profile>/` los resultados, la memoria y el `run_report.json`. `store.pack_for_review(ART)` comprime la parte de `artifacts/<profile>/` sin las cachés pesadas; agregá vos `eval/`, `corpus_card.md` y `decision_log.md`.

No incluyas el corpus crudo, los embeddings ni las cachés de los modelos.

5.2 · Rúbrica

Criterio	Qué se evalúa
Priorización	La calidad de los recortes hechos bajo una restricción real; la coherencia entre el presupuesto fijado al principio (tiempo, memoria de la T4, llamadas al LLM), lo que elegiste y lo que dice el registro de decisiones.
Diseño experimental	La vara de medir -- cómo se escribieron los ítems, el chequeo de contaminación, la granularidad del gold, congelada antes de ajustar; una variable por comparación; presupuestos igualados en tokens tal como los contó el servidor; una cota de ruido declarada antes de la afirmación.

Criterio	Qué se evalúa
Comprensión conceptual	Por qué está cada pieza y dónde actúa (offline / online, camino de escritura / camino de lectura, superficie de recuperación / superficie de generación); cómo compiten el corpus y la conversación por una sola ventana.
Lectura de resultados	Tablas leídas contra la cota de ruido; la caja abierta en los movimientos más grandes; "no llegó" distinguido de "llegó y no se usó"; el régimen declarado en cada afirmación sobre memoria.
Contribución original	Un hallazgo sobre datos sucios, una comparación o un modo de falla que nadie pidió, un vínculo con otro paper o con otra materia; una opinión que argumentás en lugar de reportar.
Reproducibilidad y claridad	Los notebooks corren a partir del conjunto de evaluación y la corpus card commiteados; los artefactos se regeneran o se verifican al cargar; los resultados por ítem nos permiten recalcular tus tablas; el informe es legible.

Cero puntos por tipear código. Un proyecto que hace menos etapas, bien razonadas, le gana a uno que las hace todas a medias, y un proyecto con algo propio para decir les gana a los dos.

---

6 · Restricciones y logística

6.1 · Reglas

- **Solo código abierto, sin APIs pagas:** `sentence-transformers`, `bm25s` + PyStemmer, `transformers` (solo para tokenizers y plantillas de chat), numpy, pydantic, y Ollama corriendo `qwen3` localmente. LangChain / LangGraph / LlamaIndex / RAGAS son opcionales (Etapa 7).
- **Declarale todo a los frameworks.** Todos usan por defecto un modelo o un embedder pago cuando no decís otra cosa: declará explícitamente el LLM local, el embedder local y `num_ctx`. Un framework que te pide una API key eligió en silencio un valor por defecto.
- **Datos:** de corpora.md, o los tuyos. Fijá la versión, respetá la licencia y su atribución, y nunca commitees los datos crudos.
- **Los escenarios de memoria son ficticios:** nada de datos personales reales en tus guiones.
- **Grupos:** hasta 4 integrantes.

6.2 · Ollama: lo que tenés que saber del servidor

- **La API de chat no tiene estado.** Tu cliente manda la conversación entera en cada llamada.
- **`num_ctx` es un parámetro por llamada.** Si no lo mandás, el servidor usa su valor por defecto: **4.096 tokens en una T4**. Las versiones recientes eligen ese valor según la memoria de la GPU, y los 15 GB de una T4 dan 4.096. La referencia online del Modelfile dice 2.048: medilo, no confíes en la documentación.
- **El endpoint `/v1` compatible con OpenAI no permite fijar `num_ctx` por llamada**, en absoluto. Usá la API nativa y mandá `num_ctx` en cada llamada (`llm.chat` lo hace).
- **Qué hace el servidor cuando un prompt no entra** lo lista el docstring de `llm.py`, medido en la versión que fija este lab (Ollama 0.34.4, `llm.OLLAMA_VERSION`). En Colab, `llm.setup` instala esa versión; en local instalás la que quieras, y `llm.setup` avisa si no coincide. Como parte de ese comportamiento cambia entre versiones, `llm.chat` lo verifica en cada llamada sobre tu versión. La Etapa 4 te hace verlo suceder una vez, a propósito.

- **Fijá los tags de los modelos**; nunca un `qwen3` a secas.

6.3 · Colab

- **Entorno de ejecución:** Google Colab, GPU T4 (nivel gratuito). Usá un flag `SMOKE_TEST` para iterar sobre un subconjunto mínimo antes de comprometerte con una corrida completa (§7.4).
- **Importá `environment` primero**, en cada notebook. Entre otras cosas, evita que JAX (que trae `bm25s`, y que Colab tiene instalado) reserve de antemano la mayor parte de la GPU al inicializarse. Si eso pasa, Ollama corre el generador en silencio parcialmente en la CPU, varias veces más lento, sin ningún error.
- **Después de reiniciar el entorno de ejecución, chequeá dónde corre el modelo:** `llm.placement(model)` debería decir `100% GPU`. Un proceso del modelo que quedó huérfano de un entorno anterior puede conservar su VRAM a través de un reinicio; *Disconnect and delete runtime* (*Desconectar y borrar el entorno de ejecución*) lo libera.
- **El disco de Colab es efímero: planificá en consecuencia.** Cuando el entorno de ejecución se desconecta (unos 90 minutos inactivo, 12 horas como máximo en el nivel gratuito), todo lo que hay bajo `artifacts/` desaparece, salvo que esté en Drive. Este es un trabajo de tres semanas; no lo vas a hacer en una sola sesión. Poné `USE_DRIVE = True` en la primera celda de cada notebook, y dejá que las etapas reusen lo que ya está construido. Los modelos en sí (unos 5 GB el generador, 3 GB el embedder y el reranker) se vuelven a descargar en cada VM nueva: presupuestá unos minutos para eso.
- **Llevar el código a Colab:** la primera celda de cada notebook clona el repo, hace `cd` adentro, trae los arreglos e instala los requirements. Para tu propio código, ver la sección "Trabajar desde el fork de tu grupo" del README.

---

7 · Guía de dimensionamiento

Rangos de partida. Elegí valores concretos que entren en tu presupuesto: esa elección es en sí parte de lo que se evalúa. Son rangos, no metas; contá los tuyos en lugar de confiar en los nuestros.

7.1 · Corpus e índice

- **Subconjunto del corpus. No** tiene que entrar en la ventana, y por un margen amplio: de unos cientos de miles a unos pocos millones de tokens del generador (aproximadamente 50-300 documentos largos, o unos pocos miles de posts o bloques de discurso). Tiene que incluir los vecinos confundibles que querés poner a prueba.
- **Hijos y padres.** Hijos de 200-400 tokens en el tokenizer **del reranker**, prefijo incluido, con 10-20% de solapamiento; chequeá el par (consulta + hijo) más largo contra la ventana de 512 tokens del reranker. Padres = la unidad estructural, con un tope de alrededor de 1.000-1.500 tokens del generador.
- **Embedder.** `BAAI/bge-m3` en fp16, `max_seq_length` 512-1.024. *Declara* 8.192, pero trunca en silencio en lo que sea que fijes: contá en su propio tokenizer. No usa prefijos de *instrucción* de consulta/pasaje (a diferencia de la familia e5); no confundir con el prefijo contextual de la Etapa 3, que es parte del texto del pasaje.
- **Reranker.** `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1` (multilingüe), `max_length` 512, corrido sobre los hijos antes de subirlos a sus padres. Un reranker de MS MARCO solo en inglés empeora el español.
- **Primera etapa y entrega.** 30-100 candidatos por canal; RRF `k = 60`; entregá 1.500-5.000 tokens del generador (o k = 3-8, y reportá los tokens).

7.2 · Generador y ventana

- **Generador:** `qwen3:8b`, `think=False`, temperatura 0, un schema, un `num_ctx` explícito.
- **Por qué no el tag `qwen3:4b`.** En Ollama, ese tag resuelve a Qwen3-4B-*Thinking*-2507, una versión solo-thinking, no el hermano híbrido de `qwen3:8b` (ese es `qwen3:4b-q4_K_M`). Entonces:
  - `think=False` puede no hacer lo que esperás;
  - "4b vs 8b" pasa a comparar dos post-entrenamientos, además de dos tamaños;
  - tampoco compra mucho contexto: su caché KV cuesta exactamente lo mismo por token (la misma geometría 36 × 8 × 128, ver abajo), así que lo único que libera son sus pesos, ~2,7 GB más chicos.

  Uses el tag que uses, fijalo y verificalo con `llm.probe_thinking`.
- **La cuenta de `num_ctx`.** La caché KV de qwen3 cuesta unos 144 KiB por token en fp16 (36 capas × 8 cabezas KV × 128 dimensiones × 2 por K y V × 2 bytes):

`num_ctx`	Caché KV
4.096	~0,6 GB
8.192	~1,2 GB
16.384	~2,4 GB
32.768	~4,8 GB

  A eso sumale unos 5,2 GB de pesos y el buffer de cómputo que reserva el servidor. 8.192 es cómodo con el embedder y el reranker cargados; 16.384 es posible; más es una decisión que hay que defender. Mantené `num_ctx` fijo dentro de un experimento.
- **Reserva de salida:** `num_predict` 256-768; el prompt tiene que entrar en `num_ctx − num_predict`.

7.3 · Experimentos y evaluación

- **El experimento de la Etapa 5B.**
  - `num_ctx` fijo (8.192). Dentro de él, un total de pasajes + historial que imponés vos, dimensionado según tu historial y no según la ventana.
  - El presupuesto de historial más chico debería ser una restricción real en la mayoría de los turnos puntuados.
  - Guiones reproducidos de a uno: unos pocos cientos de tokens de historial; un presupuesto de historial mayor que eso no restringe nada.
  - Guiones reproducidos uno detrás de otro: unos pocos miles de tokens hacia el final. Con un total de pasajes + historial de alrededor de 2.000-3.000 tokens, cada división restringe en algún punto.
  - El prompt de sistema (unos pocos cientos de tokens) y la memoria (0-1.000) van aparte.
  - Mové un límite por vez.
- **Conjunto de evaluación.**
  - Un solo turno: ≥ 30 preguntas (≥ 3 fuera del corpus; ≥ 5 de identificador, donde tu corpus tenga identificadores; ≥ 5 de metadatos/temporales), con gold a nivel de sección en todos los ítems con respuesta.

    - 8-15 guiones de conversación, de 3-6 turnos.
    - 3-6 escenarios de memoria de 2-4 sesiones, con 5-10 sondas cada uno, y al menos una actualización y un hecho nunca dicho por escenario.
- **Memoria:** recuperá 3-10 recuerdos por turno, dentro de un espacio de memoria de 300-1.000 tokens; resumí el historial alrededor del 70% de su presupuesto (la línea de advertencia de MemGPT); recencia de 0,995 por hora, solo sobre timestamps simulados.
- **Juez:** veredictos estructurados; auditá ≥ 30 juicios o el 20%.
- **Llamadas al LLM:** contalas antes de correr (§2.1). A unos 10-20 segundos por respuesta fundamentada en una T4, 60 ítems × 3 brazos es del orden de una hora.

7.4 · Smoke test, chequeos de cordura y reproducibilidad

- `SMOKE_TEST = True` es una corrida mínima. Sus números no significan nada: existe para probar que el código corre y que las guardas funcionan.
    - Datos: los documentos gold de unas seis preguntas de un solo turno (una o dos por categoría) más una muestra con semilla de distractores; dos guiones de tres turnos; un escenario de dos sesiones con tres sondas.
    - Se guardan en `eval/smoke/` y se cargan con `schema.load_eval("eval/smoke", require_frozen=False)`, así nunca tocan el conjunto congelado.
    - Modelos: un embedder chico y un generador chico están bien (por ejemplo, `intfloat/multilingual-e5-small`, que espera los prefijos `"query: "` / `"passage: "`, y `qwen3:1.7b`). Registrá cuáles en el manifiesto.
    - `num_ctx` puede ser más chico, **siempre que cada presupuesto se derive de él**.
    - Incluí **un prompt deliberadamente demasiado grande y un id gold deliberadamente faltante**, para ver saltar cada guarda. Ojo: con `require_frozen=False`, `load_eval` no chequea ids; el id faltante lo atrapa `schema.validate_eval`, o tu chequeo contra el índice (§4).
- **Chequeos de cordura (no son metas):**
    - el hit@k de documento al azar es de alrededor de k / (cantidad de documentos), y el piso real es tu fila solo-BM25;
    - si a libro cerrado se responde la mitad de tus preguntas, tu conjunto está evaluando el preentrenamiento, no la recuperación;
    - `prompt_eval_count` debería coincidir con la cuenta de tu plantilla con una diferencia de pocos tokens; un valor menor significa truncamiento.
- **Reproducibilidad:** semillas para el muestreo y los bootstraps; temperatura 0; tags de modelos fijados, corpus fijados, el hash de `FROZEN.json`. Volvé a correr un puñado de llamadas: si las salidas varían, esa varianza es parte de tu piso de ruido.

Apéndice A · Glosario

- **Abrir la caja.** Mirar, pregunta por pregunta, qué hizo cada canal (los rankings de BM25 y del denso, antes de fusionarlos) en las preguntas que cambiaron de resultado. Sirve para saber *por qué* ganó una pieza, no solo *si* ganó.
- **Abstención.** El modelo dice que no puede responder con el contexto que tiene. Se mide con el booleano del schema (`can_answer`), nunca buscando una frase en el texto.
- `answer_key` **(clave de respuesta).** Cadenas cortas que una respuesta correcta debería contener.

- **Answer-delivered (respuesta entregada).** La clave de respuesta aparece en el texto que se le entregó al generador. Mide la recuperación, no la generación.
- **Brazo.** Cada una de las configuraciones que comparás en un experimento (por ejemplo, la división 80/20 vs la 50/50). Todos los brazos corren sobre los mismos ítems.
- **Camino de escritura / camino de lectura.** En la memoria (Etapa 6): qué se guarda y cómo se actualiza (escritura), y qué se recupera y se pone en el prompt (lectura).
- **Casi-duplicado.** Dos documentos con texto casi idéntico y distinto `doc_id`. Puede ser una **versión** (respuestas distintas) o una **copia** (intercambiables). Etapa 1.
- **Congelar.** Fijar el conjunto de evaluación con `schema.freeze_eval` antes de ver resultados; a partir de ahí, cualquier edición se detecta. Etapa 2.
- **Contaminación.** Una pregunta que comparte palabras o frases exactas con su pasaje gold, y que por eso mide copia en lugar de recuperación. Etapa 2.
- **Contención.** Criterio de puntaje sin juez: la respuesta cuenta como correcta si contiene alguna cadena de la `answer_key`, después de normalizar. No distingue "sí" de "no" (Etapa 2).
- **Cota (o piso) de ruido.** La diferencia más chica entre dos filas que podés distinguir del azar con tu cantidad de preguntas. Etapa 3.
- **Cross-encoder / reranker.** Un modelo que puntúa cada par (consulta, pasaje) de forma conjunta. Se usa para reordenar los candidatos que trajo una primera etapa.
- **Dependencia (de un turno).** La etiqueta (`depends_on`) que dice cómo se relaciona un turno con el historial: `standalone` (el primero), correferencia, elipsis, cambio de tema, vuelta a un tema anterior, o sin respuesta. Etapa 2.
- **Dev / test.** Dev: los ítems que mirás y ajustás mientras construís. Test: aquellos sobre los que reportás.
- **Discordante.** Una pregunta cuyo resultado cambió entre dos filas. En una comparación pareada, solo las discordantes llevan información.
- **Escenario, sesión, sonda.** Un escenario es una persona ficticia con varias sesiones, cada una con una fecha simulada. Una sonda es una pregunta, hecha en una sesión posterior, que necesita algo dicho antes. Etapas 2 y 6.
- **Fila.** Una configuración del pipeline en una tabla de resultados (por ejemplo, "denso + prefijo"). Una fila *acumulativa* agrega exactamente una pieza a la anterior.
- **Gold.** La ubicación etiquetada de la respuesta: el documento y la sección donde está. Puede ser un *conjunto* cuando hay copias intercambiables.
- **Guion (de conversación).** Una conversación de 3-6 turnos escrita a mano, con gold por turno y las respuestas del asistente ya escritas. Etapas 2 y 5.
- **hit@k.** La proporción de preguntas en las que algún gold aparece entre los primeros k resultados. Se calcula a nivel de documento o de sección.
- **Hijo / padre.** El hijo es el chunk chico que se indexa y se busca; el padre es la unidad estructural más grande que lo contiene, que es lo que se le puede entregar al generador.
- **HyDE.** Buscar con el embedding de una respuesta hipotética escrita por un LLM, en lugar del de la pregunta. Apéndice B.1.
- **Libro cerrado.** El generador responde sin recuperación, con lo que aprendió en el entrenamiento.
- **Línea de base.** El punto de comparación más simple: en recuperación, pegar los documentos (o las filas solo-BM25 y solo-denso); en memoria, reproducir el historial completo, o no tener memoria (el piso).
- **Memoria de trabajo / episódica / semántica / procedimental.** El vocabulario de CoALA (Apéndice B.4): lo que está en la ventana ahora / experiencias pasadas / conocimiento sobre el mundo y sobre el usuario / cómo actuar.

- **MRR (Mean Reciprocal Rank).** El promedio, sobre las preguntas, de 1 / (posición del primer gold): cada pregunta aporta 1 si el gold está primero, 0,5 si está segundo, y así (0 si no aparece).
- **`num_ctx` / `num_predict`.** El tamaño de la ventana, en tokens, que le pedís a Ollama / el máximo de tokens de salida. `num_predict` es la *reserva de salida*: el prompt tiene que entrar en lo que queda, `num_ctx − num_predict`.
- **Oráculo.** Una fila que recibe la entrada que escribiste a mano como ideal: la sección gold (Etapa 4) o la reescritura escrita a mano (Etapa 5). Sirve de referencia de techo, aunque en la Etapa 5 no necesariamente lo es: la reescritura humana puede perder contra la del modelo (Apéndice B.3).
- **Prefijo contextual.** Texto que se antepone a cada chunk antes de indexarlo (título, sección, versión, año) para que el chunk se entienda solo. No confundir con los prefijos de instrucción de la familia e5 (`"query: "`).
- **`prompt_eval_count`.** Cuántos tokens del prompt dice el servidor que evaluó. Si es menor que tu cuenta, hubo truncamiento.
- **Régimen.** Si el historial completo entra en la ventana o no. Toda afirmación sobre memoria lo declara. Etapa 6.
- **Reescritura (de la consulta).** Reformular el último turno para que se entienda sin historial, antes de buscar. Etapa 5.
- **RRF (Reciprocal Rank Fusion).** Fusiona varias listas rankeadas sumando `1/(k + rank)`, con `k = 60`. Apéndice B.1.
- **Smoke (`SMOKE_TEST`).** Una corrida mínima para probar que el código corre y que las guardas saltan. Sus números no significan nada. §7.4.
- **Superficie de recuperación / de generación.** Dónde puede fallar un turno: en qué se buscó (recuperación), o en qué vio el modelo y qué respondió (generación).
- **T4.** La GPU gratuita de Colab (15 GB de memoria).
- **Vara de medir.** El conjunto de evaluación. Etapa 2.
- **Veredicto (`expected_verdict`).** En una pregunta de sí/no, si la respuesta correcta es "sí" o "no". Se puntúa aparte de la clave. Etapa 2.
- **Ventana (de contexto).** Todo lo que el modelo ve en una llamada: prompt de sistema, historial, pasajes y pregunta.

Apéndice B · Hechos de referencia

Afirmaciones específicas y verificables que vale la pena citar en tu informe, para argumentar con evidencia y no con intuiciones, y para contrastar con tus propios números. Cuando un resultado es la evaluación que un proveedor hizo de su propio producto, se aclara.

B.1 · El pipeline de recuperación y sus piezas *(Etapa 3)*

- **RAG.** Un recuperador le pasa pasajes a un generador en tiempo de inferencia. El conocimiento paramétrico queda en los pesos; el no paramétrico, en un índice que podés inspeccionar y actualizar. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020 (arXiv:2005.11401).
- **Reciprocal Rank Fusion (RRF).** Puntúa un documento con `Σ 1/(k + rank)` sobre las listas rankeadas en las que aparece, con `k = 60` en el paper original: una constante, no algo para ajustar con 30 preguntas. Cormack, Clarke & Büttcher, SIGIR 2009.

- **Cross-encoders.** Vuelven a puntuar pares (consulta, pasaje) de forma conjunta, y se usan como segunda etapa sobre una lista de candidatos de una primera etapa. Nogueira & Cho, *Passage Re-ranking with BERT* (arXiv:1901.04085).
- **HyDE.** Un LLM escribe un *documento hipotético* (un pasaje que respondería la pregunta), y se busca con su embedding en lugar del de la pregunta. El paper promedia varios de esos documentos junto con la pregunta misma. Gao et al., *Precise Zero-Shot Dense Retrieval without Relevance Labels*, ACL 2023 (arXiv:2212.10496).
- **Contextual retrieval.** Antepone a cada chunk un contexto específico, escrito por un LLM para ese chunk, antes de calcular el embedding y antes de armar el índice BM25. En la evaluación propia de Anthropic, la tasa de fallas de recuperación en el top-20 bajó de 5,7% a 3,7% solo con embeddings contextuales, a 2,9% agregando BM25 contextual, y a 1,9% agregando un reranker. Anthropic, *Introducing Contextual Retrieval* (2024). Es la evaluación de un proveedor, con su propio modelo escribiendo el contexto. Un contexto escrito por una plantilla a partir de metadatos (el prefijo de la Etapa 3) es otra cosa, mucho más barata.

B.2 · La ventana es un presupuesto, no una capacidad *(Etapas 4-5, §2.5)*

- ***Lost in the Middle*.** La exactitud tiene forma de U según la posición del documento relevante: es máxima al principio o al final del contexto. Con 20 documentos, GPT-3.5-Turbo con la respuesta en el medio sacó un puntaje *por debajo* de su exactitud a libro cerrado (56,1%). En un experimento separado de recuperador-lector, leer 50 documentos recuperados en vez de 20 mejoró la exactitud solo en alrededor de 1,5 puntos, aunque el recuperador seguía encontrando más respuestas. Liu et al., TACL 2024 (arXiv:2307.03172).
- **RULER.** De los 17 modelos de contexto largo evaluados, todos declarando 32K tokens o más, solo alrededor de la mitad mantuvo un desempeño satisfactorio a 32K. Hsieh et al., COLM 2024 (arXiv:2404.06654).
- **Qwen3 también se degrada.** El reporte de Qwen3 le da a Qwen3-8B (sin thinking) un puntaje RULER de 96,3 a 4K tokens, 91,2 a 32K y 77,4 a 128K. Es la evaluación del proveedor, sobre los pesos en precisión completa; lo que corre en una T4 es una cuantización de 4 bits. Tomá esos números como un techo, y medí los tuyos. Qwen Team, *Qwen3 Technical Report* (arXiv:2505.09388).
- ***Context Rot*.** En 18 modelos, el desempeño se vuelve cada vez menos confiable a medida que crece el largo de la entrada, aun con la tarea fija. Reporte técnico de Chroma (2025): un reporte de la industria, sin revisión de pares.
- Los hechos del servidor que vas a usar (Ollama) están en §6.2.

B.3 · Recuperación dentro de una conversación *(Etapa 5A)*

- **MTRAG**, un benchmark de RAG conversacional multi-turno (110 conversaciones, 842 tareas). Mandarle al recuperador la conversación completa, o incluso unos pocos turnos, "consistently under-performed" frente al último turno del usuario solo, y reescribir el último turno con un LLM mejoró sobre eso. Los turnos tardíos, las preguntas sin respuesta y las preguntas que no se sostienen solas fueron las más difíciles. Katsis et al., TACL 2025 (arXiv:2501.03468).
- **Reescribir, y después responder.** Separar el QA conversacional en *reescribir la pregunta, y después responderla* funciona, y el reescritor puede acercarse a la calidad humana. Vakulenko et al., WSDM 2021 (arXiv:2004.14652).
- **La reescritura humana no es necesariamente la mejor consulta.** Las reescrituras humanas "may lack sufficient information for optimal retrieval". El paper nombra cuatro propiedades de una buena

reescritura (corrección, claridad, informatividad, no redundancia) y juzga las reescrituras por la recuperación que producen. Ye et al., Findings of EMNLP 2023 (arXiv:2310.09716).

B.4 · Memoria *(Etapa 6)*

- **Un vocabulario (CoALA).** Memoria **de trabajo** (*working*: la información activa para la decisión actual; cada prompt se arma a partir de ella), **episódica** (*episodic*: experiencias pasadas), **semántica** (*semantic*: conocimiento sobre el mundo y, acá, sobre el usuario), **procedimental** (*procedural*: cómo actuar, en los pesos o en tu código y tus prompts). Sumers et al., *Cognitive Architectures for Language Agents* (CoALA), TMLR (arXiv:2309.02427). Usá estas palabras en tu informe.
- **Generative Agents** recupera recuerdos por `recency + importance + relevance` (recencia + importancia + relevancia), cada uno normalizado min-max a [0, 1], todos con peso 1. La recencia decae como 0,995 por hora de juego *desde la última vez que se recuperó el recuerdo*; la importancia es un entero de 1 a 10 que puntúa el LLM; la reflexión se dispara cuando la importancia sumada de los eventos recientes supera 150. Park et al., UIST 2023 (arXiv:2304.03442).
- **MemGPT** trata la ventana como un sistema operativo trata la RAM: un contexto principal fijo (instrucciones de sistema, un contexto de trabajo escribible, y una cola FIFO de mensajes encabezada por un *resumen recursivo* de los desalojados), respaldado por almacenamiento externo. Pasado un umbral de advertencia (por ejemplo, 70% de la ventana) le avisa al modelo que la memoria está bajo presión; en el umbral de vaciado desaloja mensajes por alrededor de la mitad de la ventana y los incorpora al resumen. Packer et al. (arXiv:2310.08560).
- **Mem0** escribe recuerdos con cuatro operaciones: ADD, UPDATE, DELETE, NOOP. En el benchmark LoCoMo, en su propia evaluación, **el contexto completo sacó más puntaje que Mem0** (puntaje de LLM-juez 72,90 contra 66,88), leyendo alrededor de 15× los tokens por pregunta (~26.000, la conversación entera, contra ~1.800 de recuerdos recuperados). La memoria compró costo y latencia, no exactitud, en los casos donde el historial entraba. Chhikara et al. (arXiv:2504.19413).
- **Zep/Graphiti** guarda dos líneas de tiempo por hecho (cuándo fue verdadero, cuándo se registró) y, ante una contradicción, *invalida* el hecho viejo en lugar de borrarlo. Rasmussen et al. (arXiv:2501.13956). No necesitás un grafo para esto: alcanza con unas columnas de timestamp en una fila.

B.5 · Evaluar la memoria *(Etapas 2, 6 y 7a)*

- **LongMemEval** evalúa cinco habilidades (extracción de información, razonamiento multi-sesión, razonamiento temporal, **actualizaciones de conocimiento**, **abstención**) sobre 500 preguntas. La exactitud cae en interacciones sostenidas: entre 30 y 60% para LLMs de contexto largo que leen el historial entero, en lugar de solo las sesiones que contienen la respuesta. Las sesiones con la respuesta están etiquetadas, así que la recuperación se puede puntuar sin juez, y las reglas del juez difieren por tipo (una respuesta actualizada cuenta aunque también se mencione la vieja). Wu et al., ICLR 2025 (arXiv:2410.10813).
- **LoCoMo** puntúa sus preguntas "adversarial" (sin respuesta) según si la salida contiene la cadena en inglés "not mentioned" o "no information available": un "no se menciona" correcto en español sacaría cero. Maharana et al., ACL 2024 (arXiv:2402.17753). Por eso este lab puntúa la abstención a partir de un campo de un schema, no de una cadena de texto.
- **En español no hay nada.** No encontramos ningún benchmark en español de memoria de largo plazo ni de RAG conversacional. Vas a escribir tus propias sondas (*probes*), sobre tu propio corpus (Etapa 2).

B.6 · Compresión y resúmenes *(Etapa 5B)*

- **El resumen recursivo** para diálogos largos funciona, pero en una auditoría manual de 100 muestras alrededor del 10% del contenido de los resúmenes estaba mal: hechos inventados (2,7%), relaciones incorrectas (3,2%), detalles faltantes (3,9%). Wang et al. (arXiv:2308.15022).
- **LLMLingua** comprime prompts de lo grueso a lo fino: descarta demostraciones enteras, y después tokens de poca información puntuados por un modelo de lenguaje chico. Reporta hasta 20× de compresión con poca pérdida. Jiang et al., EMNLP 2023 (arXiv:2310.05736).
- **RECOMP** comprime los documentos recuperados en un resumen corto, y puede devolver una *cadena vacía* cuando no ayudan; el lector entonces responde sin texto recuperado, desde sus pesos. Eso no es abstención: negarse a responder es una decisión aparte, y la vas a medir aparte. Xu et al., ICLR 2024 (arXiv:2310.04408).